Q&A Based Interactive Storytelling Using Generative AI

A

Dissertation Report

Submitted for partial fulfillment of the requirement
for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER ENGINEERING

Submitted by

Anjali Madan Jha

(124MTCM1008)

Under the guidance of

Dr. Vidyullata Devmane

Professor



DEPARTMENT OF COMPUTER ENGINEERING

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE
(An Autonomous Institute Affiliated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)

2025-2026

Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

CERTIFICATE

This is to certify that **Anjali Madan Jha** worked under my supervision for the M. Tech Dissertation report entitled **Q&A Based Interactive Storytelling Using Generative AI** and her work is of the level of requirement set up for the Dissertation report in **Computer Engineering** by Shah & Anchor Kutchhi Engineering College, Mumbai.

Dr. Vidyullata Devmane
Professor
Department of Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai – 400 088, India

The viva-voce / oral examination of **Anjali Madan Jha**, M. Tech. in Computer Engineering, has been held on .....

External Examiner	**Internal Examiner**	**Guide Name**
Head of Department	**Principal**	College Seal

CANDIDATE'S DECLARATION

I hereby declare that the work presented in the Dissertation report entitled **"Q&A Based Interactive Storytelling Using Generative AI"** for partial fulfillment of the requirement for the degree of M. Tech. in Computer Engineering and submitted to the Department of Computer Engineering at Shah & Anchor Kutchhi Engineering College, Mumbai, is an authentic record of my own work carried out during the period from January 2026 to June 2026 under the supervision of **Dr. Vidyullata Devmane**

The matter presented in this Dissertation report has not been submitted elsewhere in part or fully to any other University or Institute for the award of any other degree.

Signature of the student
Name: Anjali Madan Jha
Permanent Roll No.: 124MTCM1008
Date:

CERTIFICATE OF PLAGIARISM CHECK	
Name of the Student	Anjali Madan Jha
Title of the Dissertation	Q&A Based Interactive Storytelling Using Generative AI
Name of the Guide	Dr. Vidyullata Devmane
Name of the Co-Guide	–
Name of the Department	Department of Computer Engineering
Similar content (%) identified	9%
Acceptable Maximum Limit	10%
Name of the Similarity tool used for Plagiarism Report	Turnitin
Date of Verification	15 October 2026
Name & Sign of the Department Academic Integrity Panel (DAIP) Member	
Name & Sign of the Student	Anjali Madan Jha
Name & Sign of the Guide	Dr. Vidyullata Devmane
Name & Sign of the Co-Guide	–

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my guide, Dr. Vidyullata Devmane, Professor, Department of Computer Engineering, Shah & Anchor Kutchhi Engineering College, Mumbai, for her guidance, patience and steady encouragement throughout this dissertation. Her suggestions at every stage, from framing the problem to insisting that the system should run on a model trained for the task, shaped both the research questions and the design of the work.

I thank the Head of the Department of Computer Engineering, the Project Review Committee and all the faculty members for their suggestions during the periodic reviews. Their questions on the evaluation method and on the multimodal pipeline improved this work considerably. I am grateful to the Principal and the management of Shah & Anchor Kutchhi Engineering College for providing a good environment for research.

I gratefully acknowledge the authors of the open resources on which this work builds: the Llama 3.2 model, the *Thousand Stories, Hundred Genres* dataset [1], and the MLX, Ollama, FastAPI and D3.js projects. I also thank the fifteen volunteers who took part in the user study.

Finally, I thank my family and friends for their constant support and understanding, without which this work would not have been possible.

Anjali Madan Jha
124MTCM1008

ABSTRACT

Most story generators built on large language models take a single prompt and return a finished story, so the user must describe everything at the start and has little control afterwards. This dissertation presents an interactive storytelling system in which the story is built from the user's answers to a short set of questions on setting, character, theme and plot, and is then continued chapter by chapter in the direction the user chooses. The language model is Llama 3.2 3B, fine-tuned with QLoRA on 3,321 prompt and story pairs prepared from 1,000 public stories in the same format that the system uses, and it runs on an ordinary laptop without any external service. On 191 test samples from 49 stories that were not used in training, fine-tuning lowered the loss from 1.521 to 1.194 and the perplexity from 4.58 to 3.30 ($p < 0.001$). On 69 test prompts the fine-tuned model always returned only the story text, against 75% for the original model, at the same speed of about 37 tokens per second, although its chapters were shorter than requested. An earlier user study of the system with 15 participants, carried out with a pretrained Llama model before fine-tuning, gave mean ratings of 4.05 for coherence, 4.12 for creativity, 4.35 for engagement and 3.75 for consistency on a five-point scale.

Keywords: Generative AI, Large Language Models, Llama, Fine-Tuning, LoRA, Interactive Storytelling, Question Answering, Local Inference.

Table of Contents

Certificate . . . ii
Candidate's Declaration . . . iii
Certificate of Plagiarism Check . . . iv
Acknowledgements . . . v
Abstract . . . vi

List of Figures . . . **ix**

List of Tables . . . **x**

1. Introduction . . . **1**
1.1 Background . . . 1
1.2 Problem Statement . . . 2
1.3 Objectives of the Study . . . 2
1.4 Significance of the Study . . . 3
1.5 Structure of the Report . . . 3

2. Literature Review . . . **4**
2.1 Introduction . . . 4
2.2 Overview of the Topic . . . 4
2.3 Theoretical Framework . . . 5
2.3.1 Narrative Structure . . . 5
2.3.2 Transformer Language Models . . . 5
2.3.3 Low-Rank Fine-Tuning . . . 6
2.3.4 Local Inference . . . 6
2.3.5 Prompting . . . 6
2.3.6 Image and Speech Synthesis . . . 7
2.4 Previous Research . . . 7
2.4.1 Controllable and Long-Form Story Generation . . . 7
2.4.2 Interactive and Co-Creative Writing Tools . . . 7
2.4.3 Multimodal Stories . . . 8
2.4.4 Summary of Reviewed Literature . . . 8
2.5 Research Gaps . . . 9
2.6 Summary . . . 10

3. Experimental Model . . . **11**
3.1 Introduction . . . 11
3.2 Research Design . . . 11
3.2.1 Block Diagram of the Proposed System . . . 11
3.2.2 Overall Design . . . 12

3.2.3 Question-and-Answer Framework . . . 14
3.2.4 Context Projection and Prompt Construction . . . 16
3.2.5 Five-Phase Story Progression . . . 17
3.2.6 Dataset Construction . . . 17
3.2.7 Fine-Tuning of the Language Model . . . 19
3.2.8 Deployment of the Fine-Tuned Model . . . 21
3.2.9 Generation Loop . . . 22
3.2.10 Backend Modules . . . 23
3.2.11 Multimodal Pipelines . . . 25
3.2.12 Implementation . . . 27
3.2.13 Experimental Setup . . . 28
3.3 Data Analysis Techniques . . . 29
3.3.1 Evaluation Metrics . . . 29
3.3.2 Analysis Procedure . . . 30
3.3.3 Verification of the Measurement Pipeline . . . 30
3.4 Ethical Considerations . . . 30
3.5 Limitations . . . 31

4. Results and Discussions . . . 32
4.1 Results . . . 32
4.1.1 Introduction . . . 32
4.1.2 Descriptive Statistics . . . 32
4.1.2.1 Dataset . . . 32
4.1.2.2 Training Run . . . 33
4.1.2.3 Generated Text and Speed . . . 34
4.1.2.4 Deployed System . . . 35
4.1.2.5 Phase-1 User Study . . . 37
4.1.3 Inferential Statistics . . . 40
4.1.3.1 Held-Out Loss . . . 40
4.1.3.2 Prompt Adherence of Generated Text . . . 43
4.1.4 Summary of Findings . . . 44
4.2 Discussion . . . 45
4.2.1 Introduction . . . 45
4.2.2 Interpretation of Results . . . 45
4.2.3 Comparison with Previous Research . . . 47
4.2.4 Implications of the Findings . . . 47
4.2.5 Limitations of the Study . . . 48
4.2.6 Recommendations for Future Research . . . 48

5. Conclusion . . . 49
5.1 Conclusions . . . 49

5.2 Contributions to Knowledge 50
5.3 Practical Implications 50

Bibliography **52**

Appendices **56**

A. List of Publications **57**

B. Acceptance Letter **58**

C. Plagiarism Report **60**

List of Figures

Figure 3.1 Block diagram of the proposed system. 12
Figure 3.2 Research workflow followed in this dissertation. 13
Figure 3.3 Layered architecture of the proposed system. 14
Figure 3.4 Distribution of the 17 questions across the four dimensions. 15
Figure 3.5 Routing of user-added questions from the interface to the model prompt. 16
Figure 3.6 Construction of the Q&A-to-story fine-tuning dataset. 18
Figure 3.7 Settings panel with the model selector: the fine-tuned model and the pretrained base. 22
Figure 3.8 Landing page with the choice of question budget. 24
Figure 3.9 Single-page question form with the custom-answer option. 24
Figure 3.10 Per-segment image synthesis pipeline. 26
Figure 3.11 Narration and ambient-music signal flows. 26

Figure 4.1 Training and validation loss during fine-tuning. 33
Figure 4.2 An opening written by the fine-tuned model in the interface, with the four choices for the next step. 36
Figure 4.3 Phase 1: a chapter in the story view with its scene illustration. 37
Figure 4.4 Phase 1: narrative quality by genre (Likert 1–5, 30 stories). 38
Figure 4.5 Phase 1: user satisfaction by feature (Likert 1–5, 15 participants). . . 38
Figure 4.6 Phase 1: share of user-study sessions in which each feature was used. 39
Figure 4.7 Phase 1: image latency against prompt length, and failure rate under parallel requests. 40
Figure 4.8 Phase 1: character-extraction quality before and after filtering. 40
Figure 4.9 Held-out loss by prompt type for the pretrained and the fine-tuned model. 41
Figure 4.10 Held-out loss of each test sample under the two models. Points below the dashed line are samples on which the fine-tuned model is better. . 42

Figure 4.11 Length of the generated segments and prompt adherence for the pretrained and the fine-tuned model. 44

List of Tables

Table 2.1 Summary of reviewed work on story generation. 9
Table 2.2 Research gaps addressed in this dissertation. 10

Table 3.1 Word-count thresholds of the five story phases. 17
Table 3.2 Composition of the fine-tuning dataset. 19
Table 3.3 Fine-tuning configuration. 20
Table 3.4 Technology stack. 27
Table 3.5 REST endpoints of the application server. 28

Table 4.1 Validation loss at the checkpoints of the training run. 33
Table 4.2 Descriptive statistics of the text generated for held-out prompts (49 openings, 20 continuations). 34
Table 4.3 Phase 1: mean Likert scores by question budget (15 participants). . . 39
Table 4.4 Held-out loss and perplexity of the pretrained and the fine-tuned model, by prompt type. 41
Table 4.5 Paired comparison of held-out loss: pretrained minus fine-tuned. . . . 42
Table 4.6 Prompt adherence of the text generated for 69 held-out prompts, with paired tests. 44

Chapter 1

Introduction

1.1 Background

Large language models such as GPT-3 [2] and LLaMA [3] can write long passages of fluent text, and story writing is one of their most popular uses. They are built on the transformer architecture [4], which lets the model relate words that are far apart in a text, and they are trained on very large text collections [5]. Earlier neural story generators could stay on topic for only a few sentences [6]; current models can keep a plot going for several pages.

The usual way of using such a model is still a single request. The user types a prompt, for example “write a fantasy story about a girl and a dragon”, and receives a finished story. All decisions about characters, setting and tone either have to be written into that one prompt or are left to the model, and once the story is written the user can only accept it or start again. Many users also find it hard to describe what they want in one go; they notice their preferences only after reading a first draft.

A question-and-answer approach changes this. The system asks the user a few short questions (where the story takes place, who the main character is, what kind of conflict and ending they want) and uses the answers to write the opening. After each chapter the user picks the direction of the next one. The user does not need to know how to write a good prompt, and the story follows their choices from start to end.

Two recent developments make it possible to run such a system on an ordinary computer. Open-weight models such as Llama 3.2 [7] are small enough to run on a laptop, and low-rank adaptation (LoRA) [8] and its quantised form QLoRA [9] allow such a model to be trained for a specific task by changing only a small part of its weights. This work uses both: the story model is a Llama model fine-tuned for the questions and prompts of this system, and it runs locally.

1.2 Problem Statement

Current story generators work from one prompt, use general-purpose models that are not trained for the way an application asks for stories, and give the user little control once writing has started. A general model also tends to add titles, comments and formatting that the application then has to remove, and it has a limited context window, so earlier details of a long story can be lost.

The problem addressed in this dissertation is to design and implement a system that collects a user's story preferences through structured questions, uses a language model fine-tuned for this purpose and running locally to turn the answers into a coherent multi-chapter story, and lets the user steer the story after every chapter.

1.3 Objectives of the Study

The objectives of this work are:

1. To design a question protocol of 17 questions on setting, character, theme and plot, with budgets of 4, 8, 12 and 17 questions, custom answers and user-added questions, and to turn the answers into structured prompts.
2. To build a dataset in the system's own prompt format, fine-tune an open-weight Llama model on it with low-rank adaptation, and run the model locally.
3. To generate coherent multi-chapter stories through a five-phase story-flow model, with a scene illustration, narration and genre-matched music for

each chapter.

4. To compare the fine-tuned model with the original model on stories not used in training, and to evaluate the system with users.

The work is limited to English fiction of short to medium length (two to six chapters of 250 to 400 words), written by one user per session. Video, collaborative writing and other languages are outside its scope.

1.4 Significance of the Study

The study shows that the language model of an interactive storytelling system can be fine-tuned and run on a laptop with 8 GB of memory. Story text is generated without a network connection, and the user's answers are not sent to any outside service. Narration and music also run in the browser; only the scene illustrations use an online image service.

It also describes a simple way to prepare training data for an application that builds its own prompts: public stories are labelled with answers to the application's questions and passed through the application's prompt code, so the model is trained on exactly the prompts it will receive. Finally, the `custom_elements` context channel lets users add their own questions, and the answers reach the model's prompt without any change to the code or the database.

1.5 Structure of the Report

Chapter 2 reviews earlier work on story generation, interactive and co-creative writing tools, model adaptation and multimodal stories, and lists the gaps this work addresses. Chapter 3 describes the proposed system, the dataset, the fine-tuning and deployment of the model, and the evaluation method. Chapter 4 presents and discusses the results. Chapter 5 gives the conclusions of the work. The appendices give the list of publications, the acceptance letter and the plagiarism report.

Chapter 2

Literature Review

2.1 Introduction

This chapter covers the work on which the proposed system builds. It starts with a short overview of automatic story generation, explains the main ideas used later in the report (transformer language models, low-rank fine-tuning, local inference, prompting, and image and speech synthesis), reviews earlier story generation systems and writing tools, and ends with the research gaps that this dissertation addresses.

2.2 Overview of the Topic

Neural story generation became practical with sequence-to-sequence models trained on large story collections. Fan et al. [6] collected about 300,000 prompt and story pairs from an online writing forum and generated a story in two steps, first a short premise and then the story conditioned on it. Yao et al. [10] first produced a storyline of key words from a title and then wrote the story from the storyline. These models produced locally fluent text, but they lost track of characters and events after a few sentences.

Large pretrained transformers improved this considerably. GPT-3 [2] could write a convincing story from a short instruction, and later open models such as LLaMA [3] made similar models available for local use. Instruction tuning [11] made models follow requests such as "write the opening of a mystery story in about 300 words" much more reliably. The main open problems are now

control over the story, coherence over many pages, and how the user takes part in writing it [5].

2.3 Theoretical Framework

2.3.1 Narrative Structure

A common way to describe the shape of a plot is Freytag's pyramid, which divides a story into five parts: exposition, rising action, climax, falling action and resolution. The proposed system uses these five parts as the phases of its story-flow model, so that a generated story moves towards a turning point and an ending instead of continuing without direction.

2.3.2 Transformer Language Models

A transformer maps a sequence of tokens to a sequence of vectors through layers of self-attention and feed-forward networks [4]. In self-attention each token forms a query Q, a key K and a value V, and the output is

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V, \tag{2.1}$$

where d_k is the size of the key vectors. The matrices that compute Q, K and V and the one that projects the result back are called the query, key, value and output projections. A decoder-only model such as Llama writes text one token at a time and is trained to minimise the cross-entropy of the next token,

$$\mathscr{L}(\theta) = -\frac{1}{N}\sum_{t=1}^{N} \log p_\theta\left(x_t \mid x_{<t}\right). \tag{2.2}$$

The exponential of this loss is the perplexity; lower values mean that the model predicts the text better. The model can only attend to a fixed number of tokens, its context window, so a storytelling system has to decide which parts of a long story to include in each request.

2.3.3 Low-Rank Fine-Tuning

Fine-tuning continues the training of a pretrained model on data for a specific task. Updating all weights of a model with billions of parameters needs a lot of memory. Low-rank adaptation (LoRA) [8] keeps the pretrained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ fixed and learns a small correction

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll \min(d,k), \tag{2.3}$$

where r is the rank and α a scaling constant. Only A and B are trained, and B starts at zero, so training begins from the behaviour of the original model. QLoRA [9] stores the frozen weights in 4 bits, which reduces memory enough to fine-tune a model of a few billion parameters on a single consumer device. The result is a small adapter file that is loaded together with the original weights.

2.3.4 Local Inference

Quantisation stores each weight in fewer bits, usually 4 instead of 16, and reduces the memory needed to run a model about four times. The llama.cpp project [12] runs such models on ordinary processors and graphics hardware using its GGUF file format, and Ollama [13] provides a local model server on top of it that can also load a LoRA adapter. MLX [14] is a framework for Apple computers that supports both running and fine-tuning quantised models. Together these tools allow a model with three billion parameters to be trained and used on a laptop.

2.3.5 Prompting

A prompt is the text sent to the model to describe the task. Liu et al. [15] survey the ways of designing prompts, such as giving the model a role, stating constraints explicitly and adding examples. The proposed system builds its prompts from fixed templates filled with the user's answers. Fine-tuning complements this: the templates define the task, and fine-tuning teaches the model to answer those particular templates well.

2.3.6 Image and Speech Synthesis

Latent diffusion models [16] generate an image by removing noise step by step in a compressed space, guided by a text description; SDXL [17] is a larger model of this kind. Pollinations.ai [18] offers access to such models through a web address, which the proposed system uses for scene illustrations. For narration, browsers provide speech synthesis through the Web Speech interface [19], and neural models such as XTTS [20] give more natural voices at a higher computing cost. Background music is taken from the royalty-free SoundHelix tracks [21].

2.4 Previous Research

2.4.1 Controllable and Long-Form Story Generation

Several studies try to give the user or the system control over the plot. Tambwekar et al. [22] used reinforcement learning to steer a story generator towards a goal event chosen in advance. PlotMachines [23] generates a story from an outline of key phrases and keeps track of which phrases have already been used. Re3 [24] writes stories of more than two thousand words by planning a premise, drafting with repeated prompts and then revising, and DOC [25] improves long-range coherence with a detailed hierarchical outline. Farrell and Ware [26] use a large language model as a guide for a classical narrative-planning search. All of these systems take their input once, as a goal, a premise or an outline, and then write without further input from the user.

2.4.2 Interactive and Co-Creative Writing Tools

Other work places the user inside the writing process. AI Dungeon [27] lets a player type any action and has a language model continue the story. Wordcraft [28] is a text editor in which writers ask a model to continue, rewrite or elaborate parts of their own story. TaleBrush [29] lets the writer draw the rise and fall of a character's fortune as a line that the model then follows. Dramatron [30] builds scripts from a one-line summary through characters and scenes down to dialogue, and was tested with theatre and film professionals. These

tools give the user strong control, but they expect free-form writing or editing from the user, and they use general-purpose models that are not adapted to the tool.

2.4.3 Multimodal Stories

Intelligent Grimm [31] generates a sequence of images for a given story with a latent diffusion model and keeps the characters consistent across the images. It does not write the story or take input from the user, and it does not add narration or sound.

2.4.4 Summary of Reviewed Literature

Table 2.1 summarises the reviewed systems.

Table 2.1: Summary of reviewed work on story generation.

#	Year	Work	Method	Limitation for this work
1	2018	Fan et al. [6]	Premise, then story, from a prompt	No user input after the prompt
2	2019	Yao et al. [10]	Storyline planning, then writing	Fixed input (title)
3	2019	Tambwekar et al. [22]	Reinforcement learning towards a goal	Goal fixed in advance
4	2020	Rashkin et al. [23]	Outline-conditioned generation	Outline given up front
5	2022	Yang et al. [24]	Plan, draft and revise long stories	No user interaction
6	2022	Yuan et al. [28]	Co-writing editor with a language model	Free-form, user must write
7	2022	Chung et al. [29]	Sketch-based control of the plot	No structured preferences
8	2023	Yang et al. [25]	Detailed outline control	Outline given up front
9	2023	Mirowski et al. [30]	Hierarchical script writing	Text only, general model
10	2024	Liu et al. [31]	Images for a given story	No story writing or interaction
11	2025	Farrell and Ware [26]	Language model guiding a planner	No user interaction

2.5 Research Gaps

The review points to five gaps that this work addresses.

1. Story generators take their input once, as a prompt, a goal or an outline [6, 10, 23, 24]; none collects the user's preferences through a structured set of questions.
2. Where the user can steer the plot, the control is set before writing starts [22, 25], or it requires the user to write and edit the text [28, 29].
3. The interactive tools use general-purpose models with prompts alone [28,

30]; none adapts a model to the prompts of its own application, and none runs the model on the user's machine.

4. Story images have been studied separately from story writing [31]; the reviewed systems do not combine text, illustration, narration and music.
5. None of the reviewed systems lets the user add new questions of their own and have the answers used in the story.

Table 2.2 shows how and where each gap is addressed in this dissertation.

Table 2.2: Research gaps addressed in this dissertation.

Gap	**How it is addressed**	**Section**
No structured Q&A interaction	A protocol of 17 questions on setting, character, theme and plot whose answers are turned into the prompt	3.2.3, 3.2.4
Control fixed at the start	After every chapter the user picks one of four suggested directions or writes their own	3.2.9
No task-specific model	Llama 3.2 3B fine-tuned on prompts produced by the system's own code, run locally	3.2.6–3.2.8
No combined text, image and sound	Scene illustration, narration and genre-matched music for each chapter	3.2.11
No user-added questions	Custom answers and user-added questions passed to the model through the `custom_elements` channel	3.2.4

2.6 Summary

Story generation has moved from small sequence-to-sequence models to large instruction-tuned transformers, and recent work has improved control and coherence over long texts. Interactive tools let users shape a story, but they expect the user to write, and they rely on general models used through prompts. Low-rank fine-tuning and local inference now make it possible to adapt a small open model to one application and run it on a laptop. The next chapter describes a system that combines these ideas.

Chapter 3

Experimental Model

3.1 Introduction

This chapter describes the experimental model of the study. It presents the design of the Q&A based interactive storytelling system and of the language model at its centre, a Llama 3.2 model fine-tuned for the system's own prompts. The chapter first explains the research design: the overall architecture, the question framework, the way answers are turned into a story context and a prompt, the five-phase story model, the construction of the training data, the fine-tuning and deployment of the model, the multimodal pipelines and the implementation. It then defines the measures and statistical techniques used to analyse the results, and closes with the ethical considerations and the limitations of the design.

3.2 Research Design

3.2.1 Block Diagram of the Proposed System

Figure 3.1 is the block diagram of the proposed system. It has three parts. The input comes from the user: the answers to the story questions, any custom answers or user-added questions, and, after each chapter, the chosen direction for the next one. The proposed system runs on the user's machine. The Q&A module collects the answers, the context-extraction module turns them into a structured story context, and the prompt builder combines this context with the template of the current story phase, supplied by the story-flow manager,

and with the previous chapters kept in memory. The fine-tuned Llama 3.2 3B model writes the chapter, and the output formatter cleans the text and adds four suggested directions. The output is the chapter with its scene illustration, narration and background music, together with the story map and character graph. The user reads the chapter and chooses the next direction, which starts the next turn.

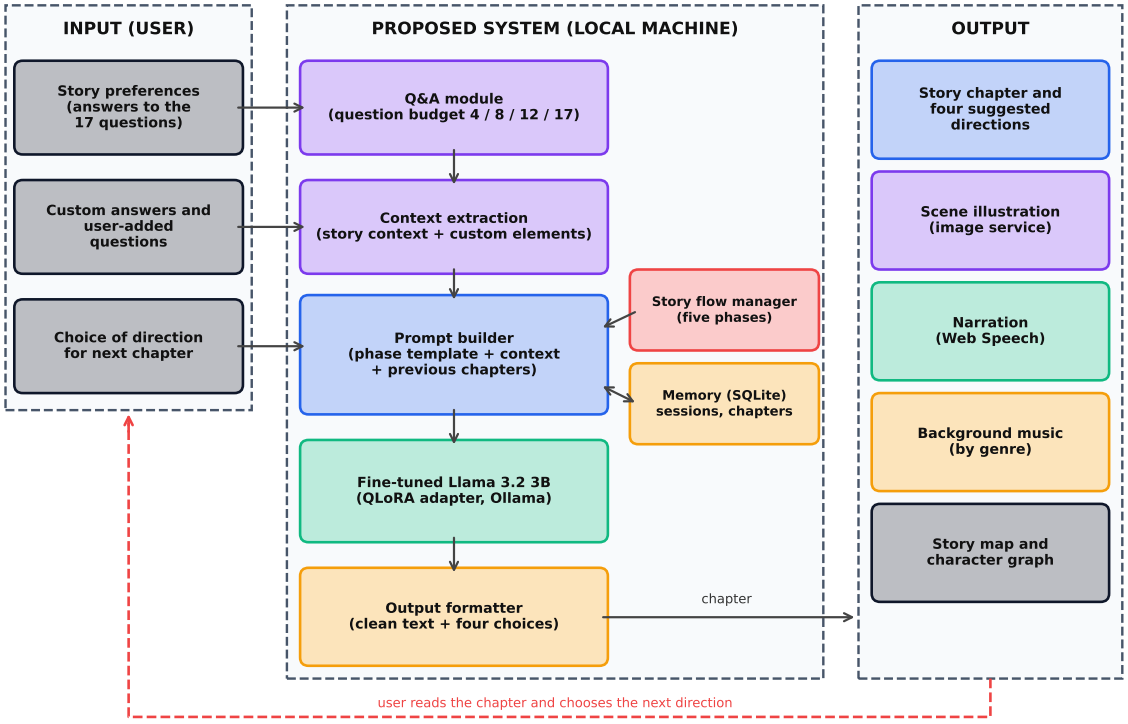


Figure 3.1: Block diagram of the proposed system.

3.2.2 Overall Design

The study follows a design-and-evaluate approach. A complete system was designed and built, a language model was adapted to it, and both were evaluated: the model by a controlled comparison with its pretrained starting point on held-out data, and the framework by a user study. Figure 3.2 shows the workflow.

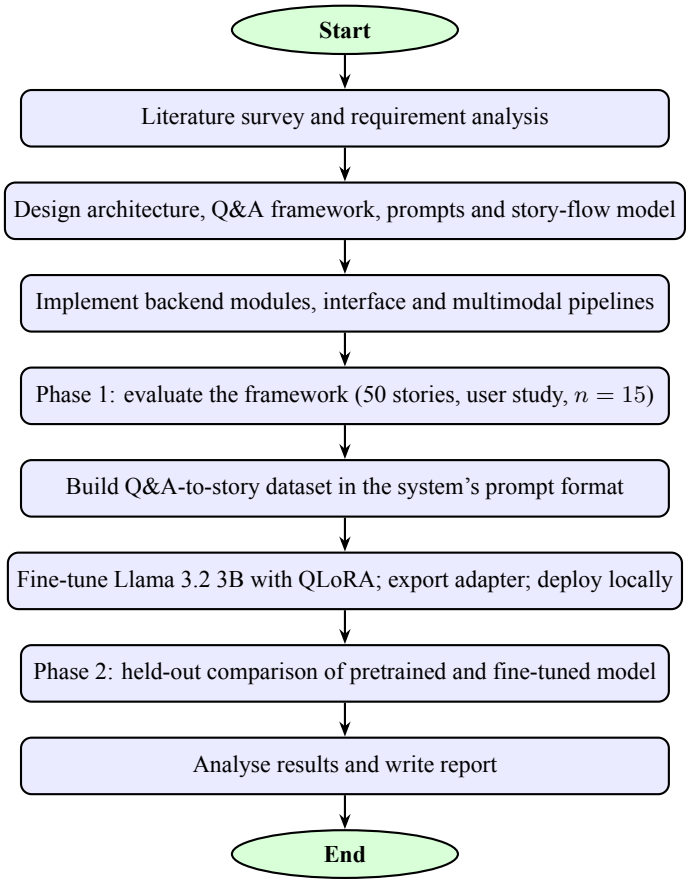


Figure 3.2: Research workflow followed in this dissertation.

The system is organised in layers, shown in Figure 3.3. The *presentation layer* is a single-page web application that shows the questions, the story, the images and the analytics, and plays narration and music. The *application layer* is a FastAPI server that exposes the REST endpoints and manages sessions. The *modular core* contains the seven backend modules that turn answers into a story. At the bottom is the *model layer*: the Ollama runtime on the same machine, which holds the pretrained Llama 3.2 3B weights and the fine-tuned adapter. Each layer depends only on the layer below it, so a module can be improved or the model replaced without touching the rest.

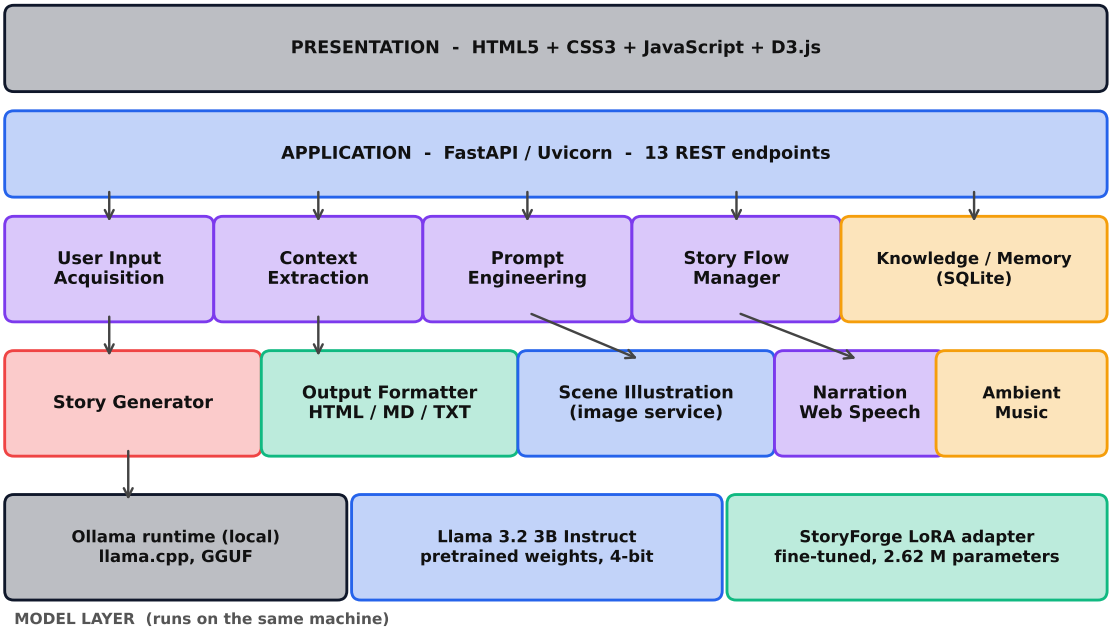


Figure 3.3: Layered architecture of the proposed system.

3.2.3 Question-and-Answer Framework

The elicitation protocol has seventeen questions divided among four narrative dimensions, as shown in Figure 3.4: four on the *setting* (kind of world, time period, atmosphere, locations), five on the *character* (the protagonist's name, personality and motivation, the allies, and the antagonist), four on the *theme* (genre, emotional journey, themes and tone) and four on the *plot* (type of conflict, plot style, complexity and ending).

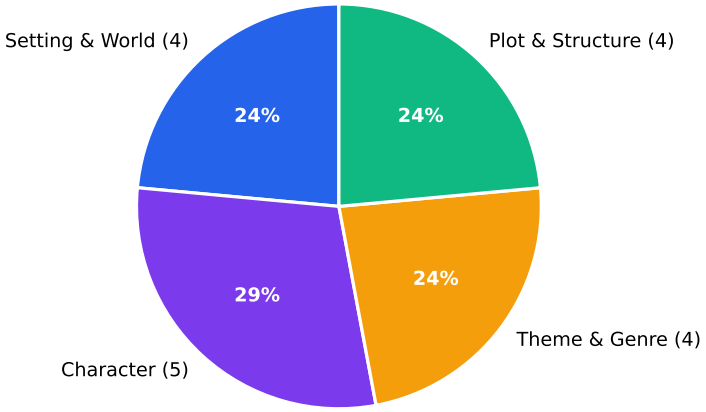


Figure 3.4: Distribution of the 17 questions across the four dimensions.

The user selects a budget of 4 (Quick), 8 (Balanced), 12 (Detailed) or 17 (Full) questions. The questions are presented in a fixed order, setting first, then character, theme and plot, and a budget of N uses the first N of them. A budget of 4 therefore covers the setting, 8 adds the protagonist and the allies, 12 adds the antagonist and three theme questions, and 17 covers all four dimensions. Attributes that are not asked take neutral default values in the prompt. Every question offers multiple-choice options and a "Write my own" option for a free-text answer. In addition, the user may add entirely new questions with their own answers.

Let $\mathscr{Q} = \{q_1, \ldots, q_{17}\}$ be the ordered question pool. The user selects a budget $N \in \{4, 8, 12, 17\}$ and gives an answer a_i to each question q_i with $i \leq N$. The user may also add M new question-answer pairs $(\tilde{q}_j, \tilde{a}_j)$, which receive the identifiers custom_q_j. The complete response set is

$$R = \{(q_i, a_i)\}_{i=1}^{N} \cup \{(\text{custom_q_}j,\ \tilde{a}_j)\}_{j=1}^{M}, \tag{3.1}$$

and it is sent to the context-extraction module as a JSON dictionary.

3.2.4 Context Projection and Prompt Construction

For each pair (q_i, a_i) the context-extraction module applies a fixed projection π_i that writes the answer into a typed field of a `StoryContext` object, for example `setting_type`, `antagonist` or `plot_style`. The projection uses keyword dictionaries, so that a free-text answer such as "a haunted lighthouse" is still mapped to a setting type. Answers whose identifiers are not known to the system, including all user-added questions, are stored word for word in a dictionary D called `custom_elements`. The final prompt P is built as

$$P = T_\phi(\mathbf{c}) \oplus \sigma(D), \tag{3.2}$$

where T_ϕ is the prompt template for the current story phase ϕ, $\mathbf{c}$ is the structured context, σ turns the dictionary D into a list of natural-language bullet points, and $\oplus$ joins the two strings. A new question added by the user thus reaches the model without any change to code or to the database schema. Figure 3.5 shows this routing.

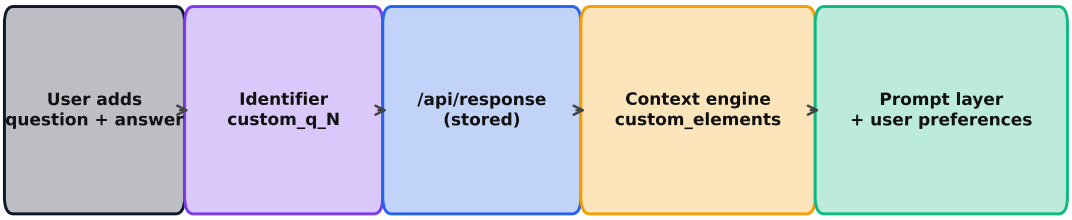


Figure 3.5: Routing of user-added questions from the interface to the model prompt.

The prompt layer has four story templates. The *init* template asks for the opening of the story (about 300 to 400 words) from the full specification. The *continuation* template asks for the next segment (about 250 to 350 words) from the recent story text and the direction chosen by the user. The *climax* and *resolution* templates ask for the turning point and the ending. A fifth template asks the model for four short choices that are offered to the user after each segment. Every request also carries a system prompt with general writing guidelines and guidance specific to the genre and the tone.

3.2.5 Five-Phase Story Progression

The story-flow manager is a finite-state machine based on Freytag's pyramid. The story moves through five phases according to its total word count, as listed in Table 3.1. The phase and the word count together select the prompt template: the continuation template is used by default, the climax template once the story has passed 1,000 words in the rising action, and the resolution template once it has passed 1,500 words in the climax or falling action.

Table 3.1: Word-count thresholds of the five story phases.

Phase	**Word range**	**Purpose**
Introduction	0–400	Establish setting and characters
Rising action	400–1200	Develop conflict and complications
Climax	1200–1800	Main confrontation or turning point
Falling action	1800–2400	Consequences of the climax
Resolution	above 2400	Resolve open threads and conclude

The thresholds were chosen from the target length of a story and from the proportions of Freytag's model; they were not tuned on data. A story is planned for about six chapters of 300 to 400 words, that is, about 2,400 words in total. The introduction is the first chapter (up to 400 words). The rising action, which carries most of the development, is given the next two chapters (400 to 1,200 words, one third of the story), and the climax and the falling action are given about one and a half chapters each (600 words, a quarter of the story). Everything after 2,400 words belongs to the resolution. The climax prompt is requested slightly before the climax phase begins, after 1,000 words, so that the chapter that crosses the 1,200-word mark is written as the turning point.

3.2.6 Dataset Construction

A general-purpose model has never seen the prompts of this system. To teach it, a dataset was built in which every sample is a prompt exactly as the running system produces it, paired with the story text that should follow. The construc-

tion is shown in Figure 3.6 and has four steps.

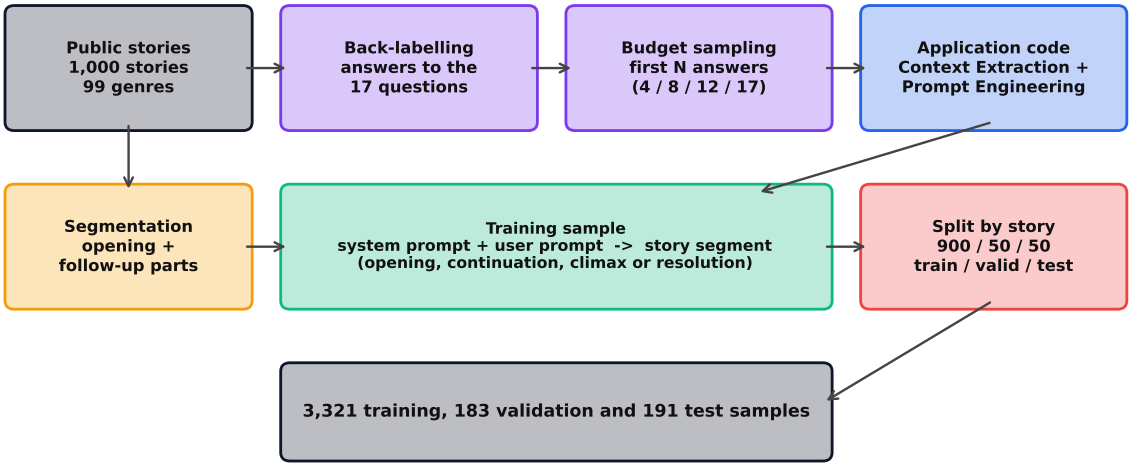


Figure 3.6: Construction of the Q&A-to-story fine-tuning dataset.

The source is the public collection *Thousand Stories, Hundred Genres* [1], released under the CC-BY-2.0 licence. It contains 1,000 short stories in English, each with a title and one of 99 genre labels, with a median length of about 980 words. Chapter headings were removed from the text, because the interface adds its own chapter titles.

Each story was given answers to the system's own questions by keyword rules. The genre label decides the genre answer and, where it is explicit, the setting and the time period. Counts of keyword groups in the story text decide the locations, the protagonist's personality and motivation, the allies, the antagonist, the themes and the ending, and the conflict type follows from the antagonist. The protagonist's name is the most frequent capitalised word that is not used as a place or a group; 53 of the 1,000 stories have no clear protagonist and keep the system's default wording.

For each story a question budget was sampled with probabilities 0.50, 0.25, 0.15 and 0.10 for 17, 12, 8 and 4 questions. The first N answers were passed through the system's own `ContextExtractionEngine` and `PromptEngineeringLayer`, the same code that serves the application. The

system and user prompts in the dataset are therefore identical to those sent at run time, including the default values used for questions outside the budget. In half of the samples the fine-grained genre label was also supplied as a user-added question, so that the `custom_elements` channel is represented in the data.

Each story was cut at paragraph boundaries into an opening of about 280 to 430 words and follow-up segments of about 230 to 360 words. The opening is the target of the init prompt. The last segment is the target of the resolution prompt, the one before it of the climax prompt when the story has at least four segments, and the others of the continuation prompt. As in the application, the continuation and resolution prompts carry the end of the two preceding segments, here limited to 350 words. The direction given in a continuation prompt is the one of the system's five continuation options that best describes the target segment.

The stories were divided at random into 900 for training, 50 for validation and 50 for testing. The division is by story, so that no segment of a validation or test story appears in training. Table 3.2 gives the composition. The longest sample has 1,285 tokens, and the median is 971.

Table 3.2: Composition of the fine-tuning dataset.

Split	Stories	Opening	Continuation	Climax	Resolution	Samples
Training	900	893	1,122	501	805	3,321
Validation	50	50	64	23	46	183
Test	50	49	71	25	46	191

3.2.7 Fine-Tuning of the Language Model

The base model is Llama 3.2 3B Instruct [7], an open-weight decoder-only transformer with 3.21 billion parameters, 28 transformer blocks, a hidden size of 3,072 and 24 attention heads. It was chosen because it is the largest recent

instruction-tuned Llama model that can be both trained and run on a laptop with 8 GB of memory.

The model was fine-tuned with QLoRA [9] using the MLX framework [14]. The base weights were kept frozen in 4-bit form. Low-rank matrices as in Equation (2.3) were attached to the query, key, value and output projections of the attention layers in the top 8 of the 28 blocks, with rank $r = 16$ and $\alpha = 32$. This gives 2.62 million trainable parameters, 0.082% of the model. The loss of Equation (2.2) was computed on the story text only: the tokens of the system and user prompts were masked, so that the model learns to write the story and not to reproduce the prompt. Table 3.3 lists the settings. Validation loss was measured every 150 steps on 25 validation samples, and the adapter was saved at each of these points.

Table 3.3: Fine-tuning configuration.

Setting	**Value**
Base model	Llama 3.2 3B Instruct, 4-bit quantised
Method	QLoRA (LoRA on frozen quantised weights)
Adapted matrices	Query, key, value, output projections
Adapted blocks	Top 8 of 28
Rank r / α / dropout	16 / 32 / 0.05
Trainable parameters	2.62 million (0.082%)
Optimiser, learning rate	Adam, 1×10^{-4}
Batch size	1, gradients accumulated over 2 steps
Training steps	600
Maximum sequence length	1,344 tokens
Loss	Cross-entropy on story tokens only
Hardware	Apple M2, 8 GB unified memory
Training time	About one hour

3.2.8 Deployment of the Fine-Tuned Model

The application runs the model through Ollama [13], which uses the GGUF format of llama.cpp [12]. The trained adapter was therefore converted to a GGUF adapter and layered on the pretrained GGUF model with a two-line model definition:

```
FROM llama3.2:3b
ADAPTER ./storyforge-lora.gguf
```

The conversion involves two points that are easy to get wrong. First, the training framework stores the two LoRA matrices transposed with respect to GGUF and uses a direct scale factor, so both matrices are transposed and α is set to the scale times the rank. Second, GGUF Llama models keep the rows of the query and key projections in a different order from the weights used in training. The same row permutation therefore has to be applied to the matrix B of every query and key adapter. This was checked on the base weights themselves before the adapter was written (Section 3.3.3). The resulting model is registered under the name `storyforge`. The adapter file is 10 MB, and the pretrained weights are shared with the base model, so both models can be offered in the interface (Figure 3.7) at no extra storage cost.

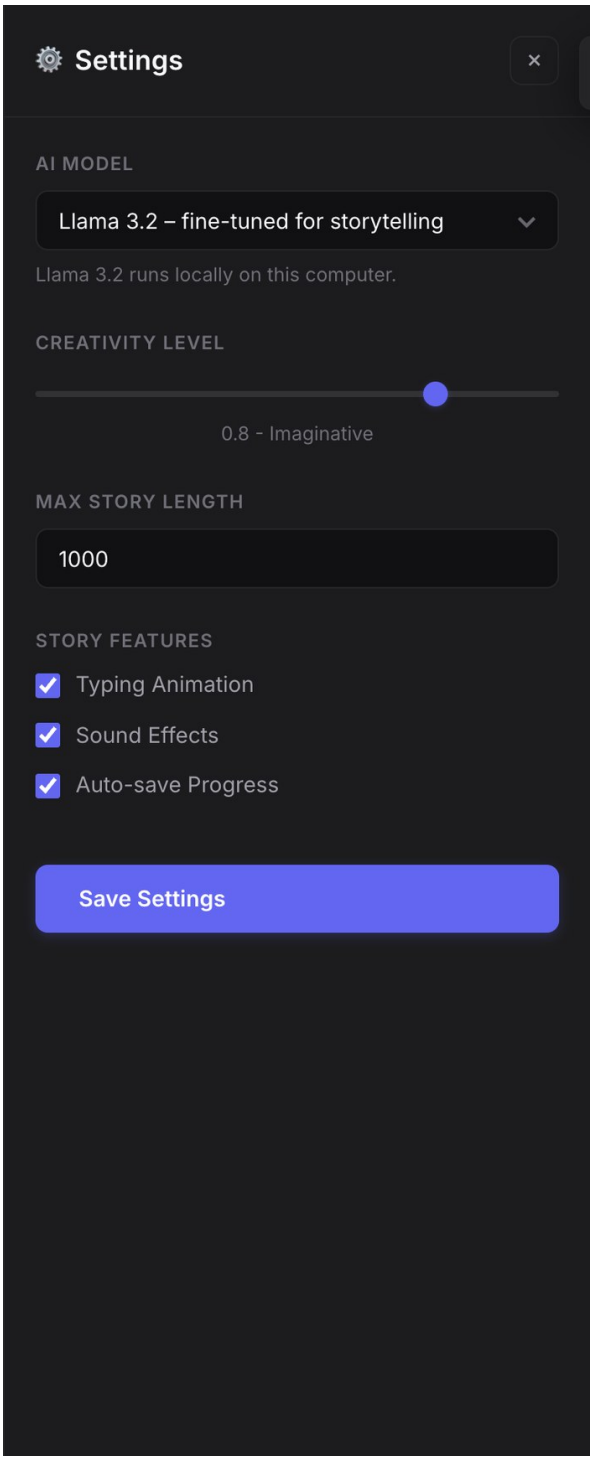


Figure 3.7: Settings panel with the model selector: the fine-tuned model and the pretrained base.

3.2.9 Generation Loop

Each turn of the story follows Algorithm 1. The model receives the system prompt and one user prompt, which carries the structured context and the end

of the two most recent segments. This keeps the request within the 4,096-token context configured for the local model while preserving the most recent events. Image generation starts as soon as the text is shown, so that the picture is often ready when the user finishes reading.

Algorithm 1 Story generation loop for one turn

Require: session s, structured context $\mathbf{c}$, custom elements D, user choice u (empty on the first turn)

Ensure: formatted segment and choices for the next turn

$H \leftarrow$ last two segments of session s
$\phi \leftarrow$ current phase from total word count
$P \leftarrow T_\phi(\mathbf{c}, H, u) \oplus \sigma(D)$
$y \leftarrow \textsc{Model}(P)$ ▷ fine-tuned Llama 3.2, local
$y \leftarrow \textsc{Format}(y)$ ▷ clean text, style dialogue
update character and plot-thread trackers with y
run continuity and pacing checks on y
save y to the database; update word count and phase
$C \leftarrow \textsc{Model}(\text{choice prompt for } y)$ ▷ four options
return y and C; the interface then requests the scene illustration

3.2.10 Backend Modules

The backend has seven modules with clear responsibilities and typed interfaces.

User Input Acquisition holds the question pool, serves it grouped by dimension and validates the answers. The landing page (Figure 3.8) lets the user choose a budget, and the question form (Figure 3.9) shows each question with its options and a "Write my own" field.

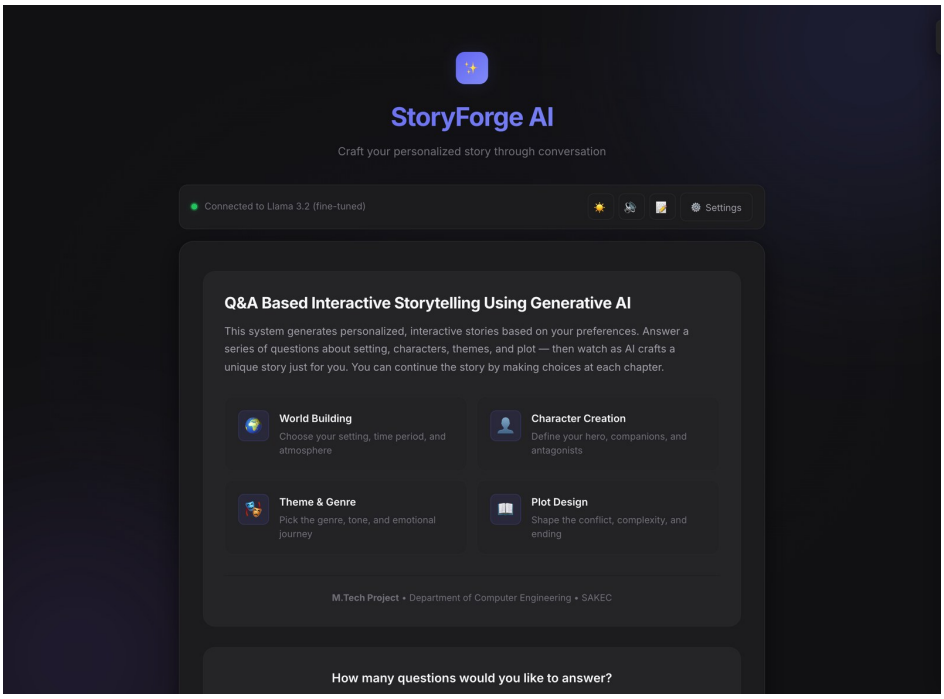


Figure 3.8: Landing page with the choice of question budget.

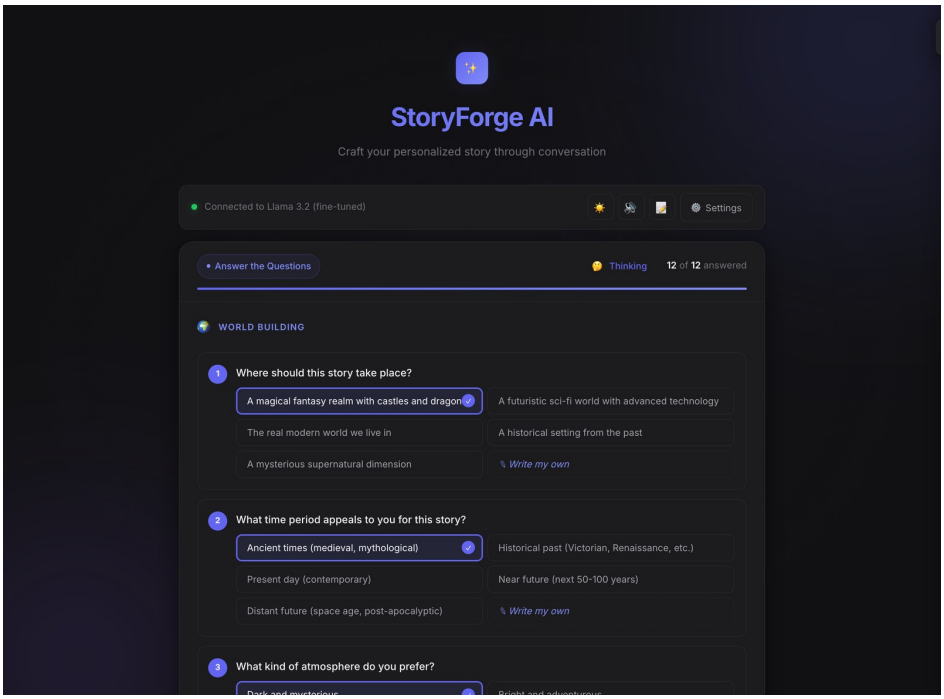


Figure 3.9: Single-page question form with the custom-answer option.

Context Extraction Engine fills the `StoryContext` object as described in Section 3.2.4. **Knowledge and Memory** stores sessions, answers, segments and preferences in SQLite and returns the most recent segments for the next

prompt. **Prompt Engineering** holds the templates and adds genre and tone guidance. **Story Generator** sends the prompt to the model through a small client interface and returns the text; the active model can be changed at run time from the settings panel. **Story Flow Manager** implements the five-phase state machine of Section 3.2.5. For every segment it records the word count, the characters and locations mentioned, the amount of dialogue and a keyword-based emotional tone, and it applies two rule-based checks: character continuity (a character who reappears after being marked as gone) and pacing (a long segment without dialogue). **Output Formatter** cleans the text, wraps dialogue in styled HTML and exports the full story as plain text, Markdown or HTML.

3.2.11 Multimodal Pipelines

For each segment, a short image prompt is built from the most visual sentence. Dialogue in quotation marks and sentences built around thought verbs are removed first, because they describe things that cannot be drawn. Each remaining sentence s is scored as

$$\text{score}(s) = 2 \cdot \mathbf{1}_V(s) - 2 \cdot \mathbf{1}_T(s) + \min\big(3,\ |\text{ProperNouns}(s)|\big), \tag{3.3}$$

where $\mathbf{1}_V(s)$ is 1 if the sentence contains a visual verb (*stood, loomed, soared, glowed, towered*, and so on) and $\mathbf{1}_T(s)$ is 1 if it contains a thought verb. The best sentence is combined with the genre, the current mood and a fixed style prefix, and the prompt is limited to 200 characters. It is sent to the Pollinations.ai image service [18], as shown in Figure 3.10. The service rejects bursts of parallel requests, so all image requests pass through a serial queue that sends one request at a time, waits 1.2 s between requests and abandons a request after 35 s. This is the only part of the system that needs a network connection and an outside service; when the service is unavailable or refuses a request, the story is shown without illustrations.

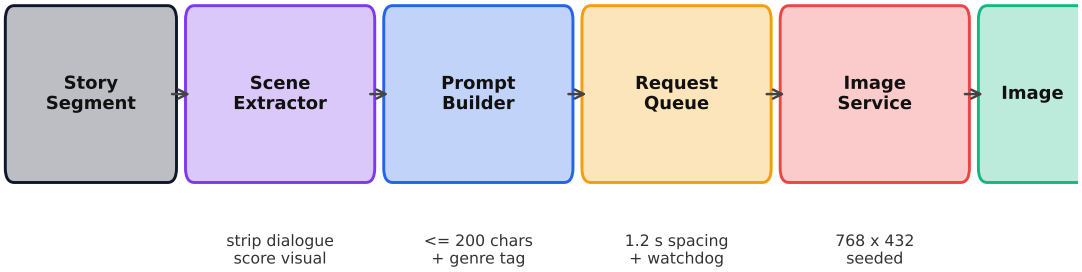


Figure 3.10: Per-segment image synthesis pipeline.

Narration uses the browser's built-in speech synthesis [19]. One utterance is queued per segment, with pause, resume and stop controls. Ambient music maps each genre to a royalty-free SoundHelix track [21], played in a loop with a 2.4 s fade-in and a 0.8 s fade-out (Figure 3.11).

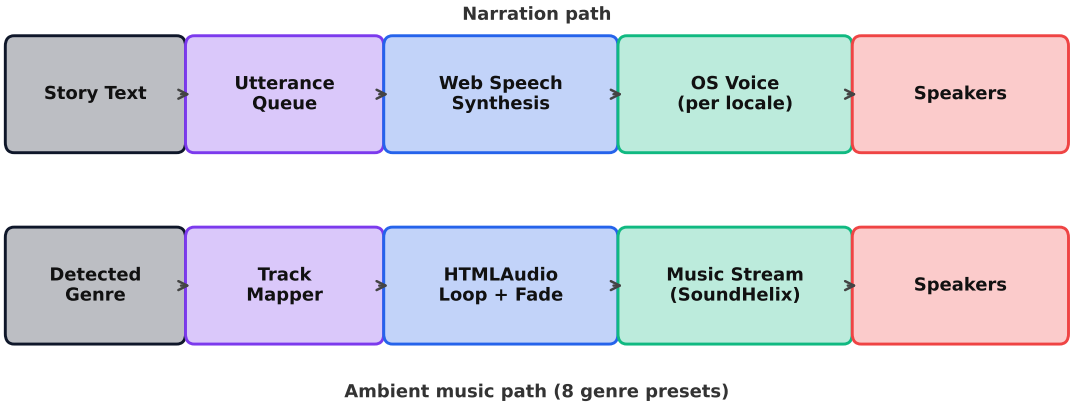


Figure 3.11: Narration and ambient-music signal flows.

The Story Map arranges the chapters as a vertical tree with the D3.js library; each node shows a preview of its chapter and the user choice that led to it. For the Character Relationship Graph, capitalised words are taken from the segment text and filtered with a stop-word list, and only names that appear at least twice are kept. The edge weight between two names is the number of segments in which both appear. Because the edges are based only on co-appearance, the interface states that the graph shows which characters appear together and not

the type of relationship between them.

3.2.12 Implementation

Table 3.4 lists the software used. All components are free, and most are open source. The SQLite database has five tables: `sessions`, `story_segments`, `story_context`, `qa_history` and `user_preferences`. The server exposes 13 REST endpoints, listed in Table 3.5, with request and response bodies typed by Pydantic [32]. The implementation has about 3,300 lines of Python in the backend, 1,700 lines of JavaScript, 2,600 lines of CSS and 300 lines of HTML in the interface, and 750 lines of Python for the dataset, export and evaluation scripts. The source code is available as open source [33].

Table 3.4: Technology stack.

Layer	**Technology**	**Purpose**
Language model	Llama 3.2 3B Instruct with LoRA adapter	Story generation
Model runtime	Ollama (llama.cpp, GGUF)	Local inference
Fine-tuning	MLX, mlx-lm	QLoRA training and evaluation
Backend	Python, FastAPI with Uvicorn	REST interface and templates
Schemas	Pydantic v2	Request and response types
Database	SQLite3	Sessions, answers, segments
Frontend	HTML5, CSS3, JavaScript	Single-page interface
Visualisation	D3.js v7	Story Map, Character Graph
Image service	Pollinations.ai	Scene illustrations
Speech	W3C Web Speech Synthesis	Narration
Ambient audio	HTMLAudio with SoundHelix	Genre music

Table 3.5: REST endpoints of the application server.

Endpoint	Method	Purpose
`/`	GET	Web interface
`/api/status`	GET	Check that the model is available
`/api/session/new`	POST	Create a new session
`/api/questions/all`	GET	All questions, grouped by dimension
`/api/question/next`	GET	Next question (sequential mode)
`/api/response`	POST	Submit an answer
`/api/story/generate`	POST	Generate the first segment
`/api/story/continue`	POST	Continue from a user choice
`/api/story/full/{id}`	GET	Full story of a session
`/api/story/download/{id}`	GET	Export as TXT, MD or HTML
`/api/settings`	POST	Change model or temperature
`/api/session/{id}/summary`	GET	Session statistics
`/api/session/{id}`	DELETE	End a session

3.2.13 Experimental Setup

The evaluation was carried out in two phases, and the results are kept apart throughout this report.

Phase 1 evaluated the framework. During its development, 50 stories were generated across six genres and all four question budgets, and a user study was conducted with 15 participants (8 male, 7 female, aged 22 to 35, with technical and non-technical backgrounds). Each participant created two stories, giving 30 stories, and rated each on a five-point Likert scale for coherence, creativity, engagement and consistency, together with their satisfaction with each feature. The character extractor was evaluated against 50 hand-annotated segments, and the image queue was tested under parallel load. In this phase the fine-tuned model did not yet exist, and the language model was the pretrained Llama 3.1 8B Instruct model, used as a hosted model. The ratings of this phase therefore describe the framework with a pretrained model, and they are not claimed for the fine-tuned model.

Phase 2 evaluated the fine-tuned model, which was compared with its pre-trained starting point, Llama 3.2 3B Instruct, in two ways. First, the loss of both models was computed on all 191 samples of the held-out test set. Second, both models generated story text, through the same runtime and with the application's own settings (temperature 0.8, top-p 0.9, at most 1,000 new tokens, a 4,096-token context), for 69 held-out prompts: all 49 opening prompts of the test set and the first 20 continuation prompts. The same random seed was used for both models on each prompt. All phase-2 training and measurements were made on an Apple M2 laptop with 8 GB of unified memory running macOS, with no other model loaded.

3.3 Data Analysis Techniques

3.3.1 Evaluation Metrics

The following measures are used.

- **Loss and perplexity.** The cross-entropy of Equation (2.2) on the story tokens of a held-out sample, and its exponential. Totals are weighted by the number of story tokens. Lower values mean that the model assigns higher probability to the reference story.
- **Prompt-adherence measures** on generated text: the number of words; the share of outputs within $\pm 25\%$ of the requested length range; the share that begin directly with story text, with no title, heading or comment about the task; the share that mention the protagonist named in the prompt; and the share that end in a complete sentence. The ratio of distinct word pairs to all word pairs (distinct-2) is reported as a measure of lexical variety.
- **Generation speed and latency.** New tokens per second as reported by the runtime, and the time in seconds to produce a segment.
- **Likert score.** The mean rating from 1 (poor) to 5 (excellent) for each dimension of the user study.
- **Precision, recall and F_1** for character extraction, with

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{3.4}$$

3.3.2 Analysis Procedure

The two models are compared on the same samples, so paired tests are used. For the held-out loss, the difference between the pretrained and the fine-tuned model is computed for each sample. The mean difference is reported with a 95% confidence interval obtained by bootstrap resampling (10,000 resamples), together with a paired t-test, the Wilcoxon signed-rank test, which makes no assumption of normality, and Cohen's d as a measure of effect size. Several samples come from the same story and are not independent, so the analysis is repeated at the level of stories, after averaging the samples of each story. For the generated text, proportions are compared with McNemar's exact test on the paired outcomes and word counts with the Wilcoxon signed-rank test. A significance level of 0.05 is used.

3.3.3 Verification of the Measurement Pipeline

Two checks were made before the results were collected. The first concerns the conversion of the adapter described in Section 3.2.8. The query, key, value and output projection weights of two blocks were read from the GGUF base model and from the training copy of the same model and compared. For the value and output projections the correlation between the two copies was 0.99 as stored. For the query and key projections it was 0.01 as stored and 0.99 after the row permutation, which confirms both that the two files hold the same model and that the permutation applied to the adapter is the correct one. The runtime also reported loading all 64 adapter matrices. The second check concerns the application path. A complete session, from creating a session and submitting 17 answers and a user-added question to generating an opening and a continuation, was run through the REST interface with the fine-tuned model, and the interface was exercised in a browser, including the switch between the two models.

3.4 Ethical Considerations

Generated stories may contain unsuitable content if the user asks for it or if the model produces it by mistake. The base model was released with its own

safety training, but the system applies no additional content filter, and it is not intended for unsupervised use by children.

The fine-tuning data is a public dataset released under a licence that permits reuse with attribution, and it is cited where it is used. The dataset card does not document how the stories were written, so the data may itself contain machine-written text; this is stated as a limitation and not concealed. The base model is used under its community licence. Because the language model runs on the user's own machine, the answers a user gives are not sent to any third party. The image prompt for each segment, a short description of a scene, is sent to the online image service.

Participants in the user study took part voluntarily, gave informed consent and could stop at any time. No personal data beyond the story answers was collected, and results are reported only as averages. The held-out test stories were not used in training or in choosing the training settings, and the results of the two evaluation phases are reported separately so that ratings obtained with one model are not attributed to another.

3.5 Limitations

The training data comes from one collection of 1,000 stories, and the answers attached to each story were produced by keyword rules, so some of them are wrong. The model was trained once, for 600 steps, without a search over training settings. The phase-1 user study used a pretrained model, so the fine-tuned model has not been rated by users.

Chapter 4

Results and Discussions

4.1 Results

4.1.1 Introduction

This section reports the outcome of the experiments defined in Chapter 3. It opens with descriptive statistics: the fine-tuning dataset, the training run, the text generated by the pretrained and the fine-tuned model on held-out prompts, the behaviour of the deployed system, and the ratings collected in the phase-1 user study. It then presents the inferential results, in which the two models are compared on the same held-out samples with paired tests. All phase-2 figures are computed on test stories that were not used in training or in choosing the training settings. Results from the two evaluation phases are kept apart: every rating by a human participant belongs to phase 1 and was obtained with a pretrained model.

4.1.2 Descriptive Statistics

4.1.2.1 Dataset

The fine-tuning dataset contains 3,695 samples built from 1,000 stories (Table 3.2). Of the 3,321 training samples, 893 are story openings, 1,122 continuations, 501 climaxes and 805 resolutions. The openings cover all four question budgets: 460 use the full set of 17 questions, 223 use 12, 134 use 8 and 76 use 4, which follows the sampling probabilities of Section 3.2.6. A training sample has 721 words on average, of which 264 are the story segment the model has to

write. In tokens, the median sample has 971 and the longest 1,285. The test set has 191 samples from 49 stories; one of the 50 test stories was too short to give a sample.

4.1.2.2 Training Run

Figure 4.1 shows the loss during fine-tuning. Before training, the loss of the pretrained model on the validation samples was 1.476. It fell to 1.207 after 150 steps, 1.164 after 300, 1.151 after 450 and 1.107 after 600 steps (Table 4.1). The training loss, averaged over 20 steps, moved from 1.46 at the start to between 1.09 and 1.32 in the second half of the run. Most of the gain came in the first 150 steps, which is expected: the model first learns the form of the answer, that is, to begin directly with story text of the requested kind and length, and then improves more slowly. The validation loss was still falling when the run ended, so the model had not begun to overfit. The run processed 192,000 story tokens in about one hour.

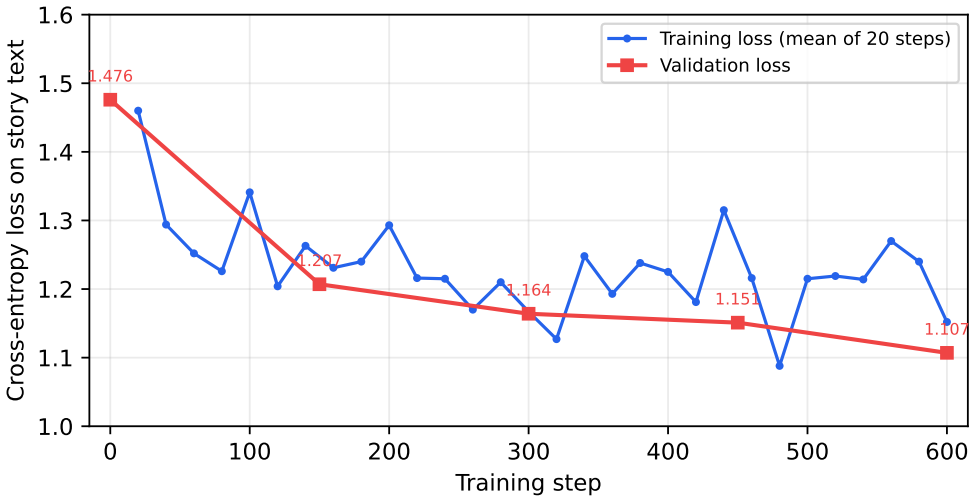


Figure 4.1: Training and validation loss during fine-tuning.

Table 4.1: Validation loss at the checkpoints of the training run.

Training step	**0**	**150**	**300**	**450**	**600**
Validation loss	1.476	1.207	1.164	1.151	1.107
Validation perplexity	4.38	3.34	3.20	3.16	3.03

4.1.2.3 Generated Text and Speed

Table 4.2 describes the text that the two models generated for the 69 held-out prompts, under the application's own generation settings.

The pretrained model wrote openings of 357 words on average, inside the requested range of 300 to 400 words, and continuations of 312 words. The fine-tuned model wrote shorter segments: 281 words for openings and 260 for continuations, and 33 of its 49 openings were below 300 words. Lexical variety was the same for both models, with about 95% of word pairs distinct. Neither model reached the limit of 1,000 new tokens in any generation.

Both models generated openings at 38.1 tokens per second. The adapter adds very little computation, so fine-tuning did not slow the model down. Because its segments are shorter, the fine-tuned model finished a segment in 10.2 s on average against 12.9 s for the pretrained model, and no segment from either model took longer than 15 s.

Table 4.2: Descriptive statistics of the text generated for held-out prompts (49 openings, 20 continuations).

	Openings (300–400 words asked)		**Continuations** (250–350 asked)	
Measure	**Pretrained**	**Fine-tuned**	**Pretrained**	**Fine-tuned**
Words, mean	357	281	312	260
Words, median	359	284	301	258
Words, range	303–418	137–366	268–375	181–328
Distinct word pairs	0.950	0.951	0.943	0.939
New tokens, mean	458	342	400	316
Tokens per second	38.1	38.1	37.8	35.3
Seconds per segment, mean	13.1	9.9	12.4	10.7
Seconds per segment, max	14.8	12.6	14.4	13.7

4.1.2.4 Deployed System

In the application, a turn consists of the story segment and a second request for the four choices. In an end-to-end session with the fine-tuned model, run through the REST interface with 17 answers and one user-added question, the opening and its choices took 28 s and the continuation and its choices 22 s. The user-added question, which asked for a silver fox as the protagonist's companion, was honoured in the opening, the continuation followed the choice that had been selected, and the choices were returned as four separate options in both turns. Figure 4.2 shows an opening written by the fine-tuned model in the interface, with the choices offered for the next step. Switching between the fine-tuned and the pretrained model from the settings panel takes effect without restarting the server.

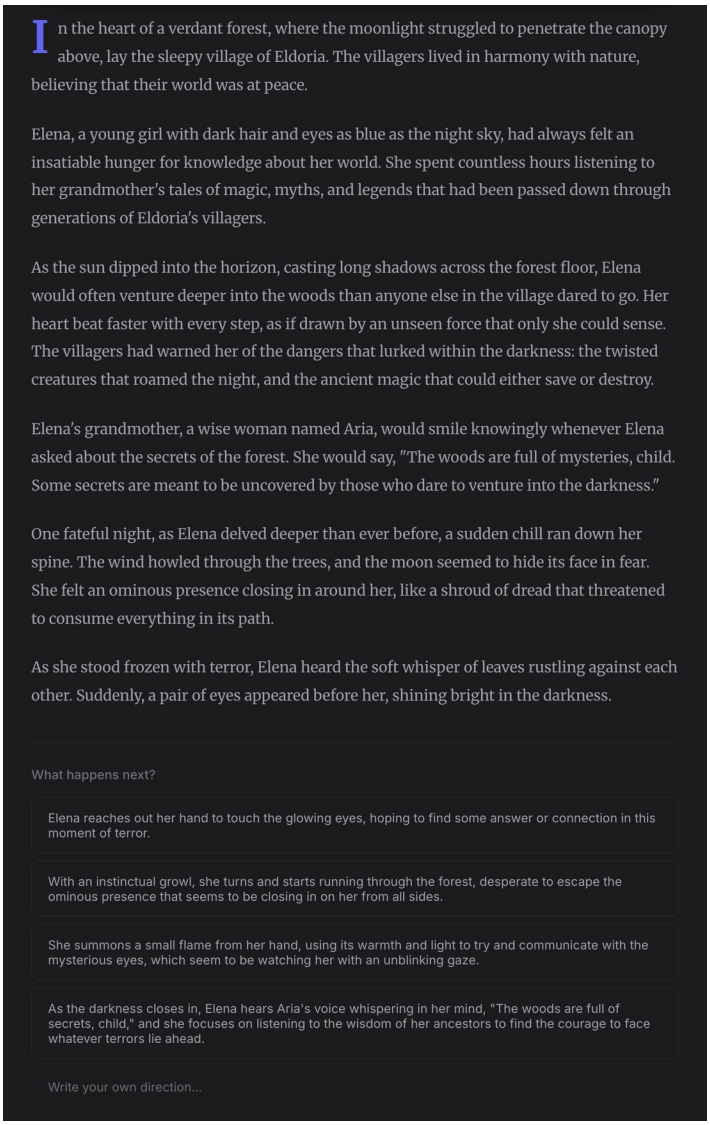


Figure 4.2: An opening written by the fine-tuned model in the interface, with the four choices for the next step.

Scene illustrations were generated throughout phase 1; Figure 4.3 shows a chapter with its illustration from that phase. During the phase-2 sessions the image service returned an image for the first few requests and then answered with a payment-required response (HTTP status 402), so the stories of phase 2 were shown without illustrations. In that case the interface displays a short notice with a retry button in place of the picture, and the rest of the story view works as before. Narration and ambient music were not affected.

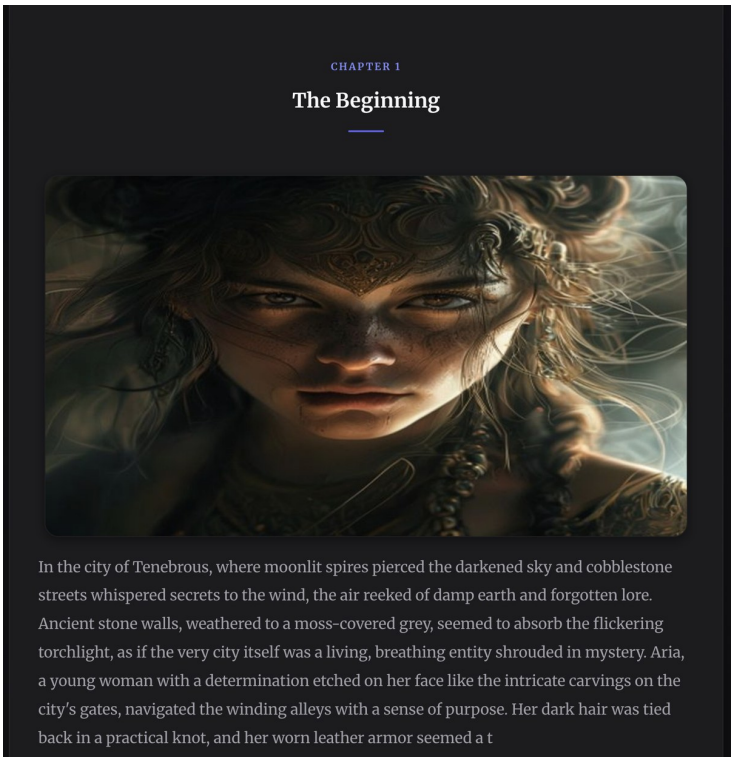


Figure 4.3: Phase 1: a chapter in the story view with its scene illustration.

4.1.2.5 Phase-1 User Study

The ratings in this part were collected in phase 1, with the pretrained Llama 3.1 8B model (Section 3.2.13). They describe the framework: the question protocol, the story-flow model and the multimodal features.

Across the 30 stories of the user study, the mean Likert scores were 4.05 for coherence, 4.12 for creativity, 4.35 for engagement and 3.75 for consistency. Figure 4.4 shows the scores by genre. Adventure received the highest engagement score (4.6). Mystery received the lowest consistency score (3.5), which is expected because a mystery depends on clues that must stay consistent across many segments.

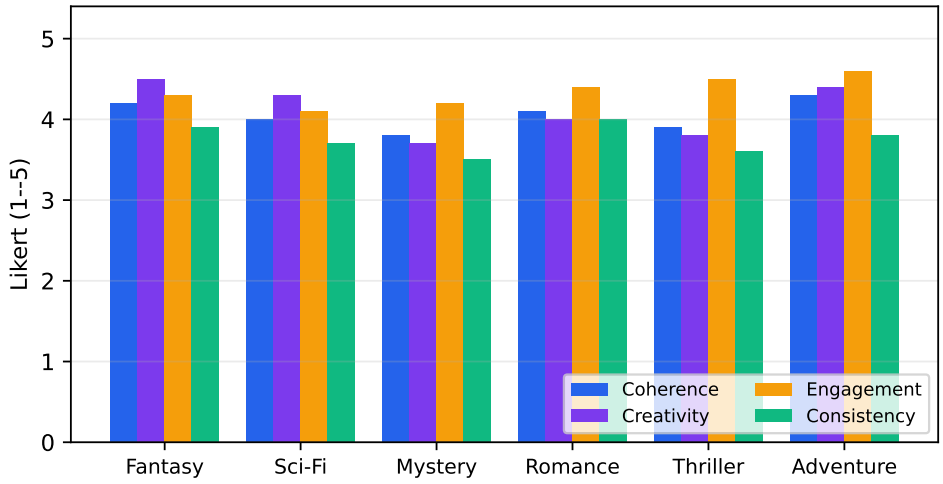


Figure 4.4: Phase 1: narrative quality by genre (Likert 1–5, 30 stories).

Figure 4.5 shows the satisfaction with individual features. The overall satisfaction was 4.3 out of 5. Custom answers (4.5) and image illustration (4.2) received the highest ratings, and the match between music and genre was rated 4.3. The lowest rating was for how meaningful the offered choices were (3.9).

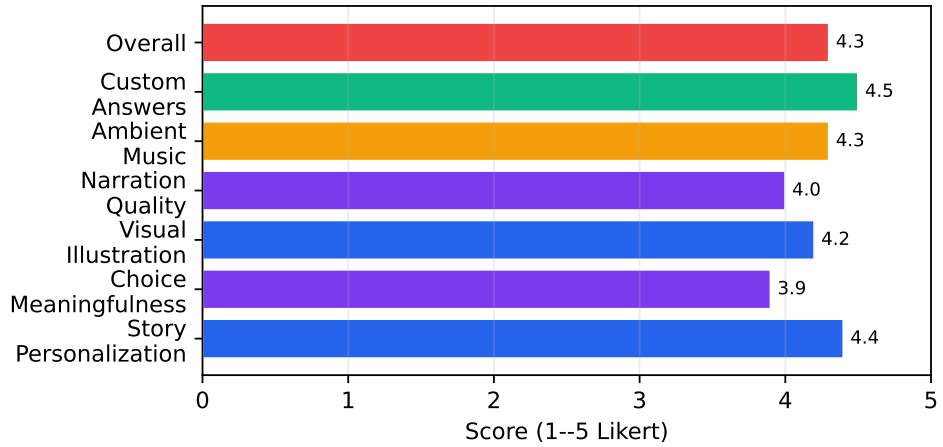


Figure 4.5: Phase 1: user satisfaction by feature (Likert 1–5, 15 participants).

Table 4.3 gives the mean scores for each question budget. Personalisation rose steadily with the number of questions, from 3.2 at 4 questions to 4.6 at 17, a gain of 1.4 points. Coherence rose more modestly, from 3.6 to 4.3. Coherence and creativity changed little beyond 12 questions, while the time spent answering kept growing.

Table 4.3: Phase 1: mean Likert scores by question budget (15 participants).

Budget	Coherence	Creativity	Personalisation	Answering time
4 (Quick)	3.6	3.8	3.2	45 s
8 (Balanced)	3.9	4.1	3.8	1.5 min
12 (Detailed)	4.2	4.2	4.3	2.5 min
17 (Full)	4.3	4.1	4.6	3.5 min

Image generation, which is on by default, was used in 91% of the user-study sessions; ambient music and custom answers, which the user has to turn on, were used in 71% and 62% (Figure 4.6).

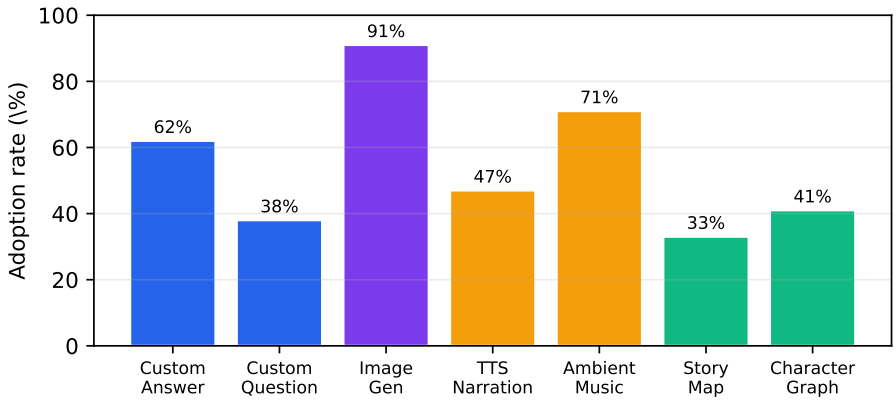


Figure 4.6: Phase 1: share of user-study sessions in which each feature was used.

Two components that do not depend on the language model were also measured in phase 1. When five image requests were sent in parallel without the queue, the failure rate rose with the length of the prompt, sharply above about 230 characters, which is why the image prompt is limited to 200 characters; with the serial queue and 1.2 s spacing, failures fell from about 70% to under 2% (Figure 4.7). For character extraction, the filtered extractor used in the system (stop-word list and a minimum of two mentions) reached a precision of 0.83 and a recall of 0.78 against 0.61 and 0.84 for a plain regular expression, so that F_1 rose from 0.71 to 0.80 (Figure 4.8).

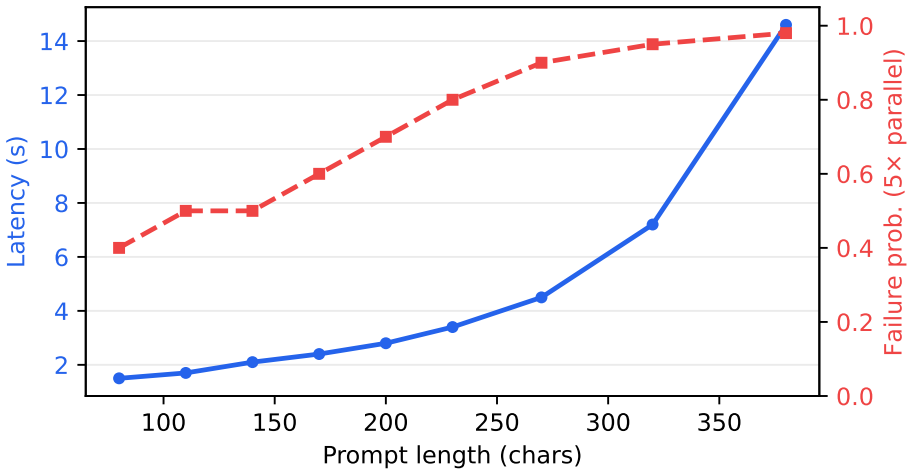


Figure 4.7: Phase 1: image latency against prompt length, and failure rate under parallel requests.

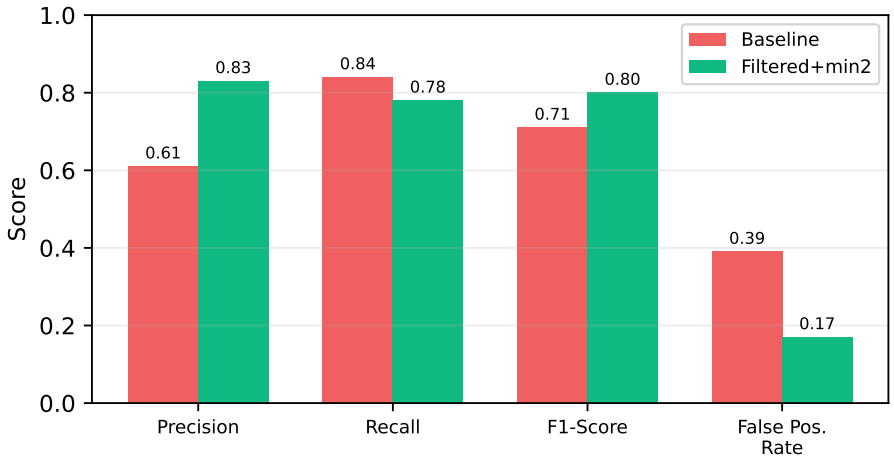


Figure 4.8: Phase 1: character-extraction quality before and after filtering.

4.1.3 Inferential Statistics

4.1.3.1 Held-Out Loss

Table 4.4 reports the loss and perplexity of the pretrained and the fine-tuned model on the held-out test set, and Figure 4.9 shows the loss by prompt type. Over all 191 samples, fine-tuning reduced the loss from 1.521 to 1.194 and the perplexity from 4.58 to 3.30, a reduction of 28%. The loss fell for every prompt type. The opening is the hardest prompt for both models, because the model has the least text to build on, and the resolution is the easiest, because most of the story is already fixed.

Table 4.4: Held-out loss and perplexity of the pretrained and the fine-tuned model, by prompt type.

			Loss		Perplexity	
Prompt type	**Samples**	**Story tokens**	**Pretrained**	**Fine-tuned**	**Pretrained**	**Fine-tuned**
Opening	49	17,547	1.705	1.370	5.50	3.93
Continuation	71	22,152	1.464	1.151	4.32	3.16
Climax	25	7,592	1.534	1.174	4.64	3.23
Resolution	46	14,188	1.377	1.052	3.96	2.87
Overall	**191**	**61,479**	**1.521**	**1.194**	**4.58**	**3.30**

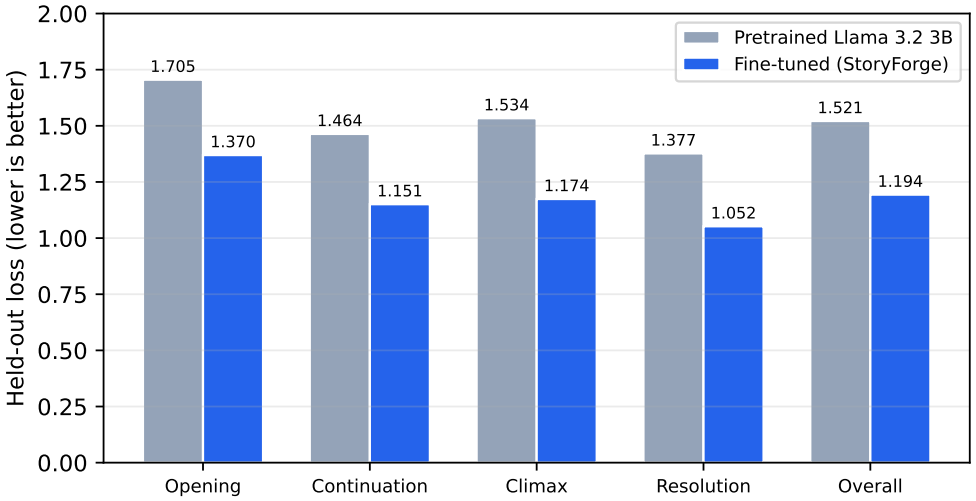


Figure 4.9: Held-out loss by prompt type for the pretrained and the fine-tuned model.

Because both models were scored on the same samples, the differences can be tested directly (Table 4.5). By sample, the mean loss fell by 0.336 (95% bootstrap confidence interval 0.327 to 0.347), and the fine-tuned model had the lower loss on every one of the 191 samples. Samples from one story are not independent, so the test was repeated on the 49 stories after averaging: the mean loss fell by 0.341 (0.326 to 0.359), and the fine-tuned model was better on all 49 stories. Both the paired t-test and the Wilcoxon signed-rank test reject the hypothesis of no difference at both levels ($p < 0.001$ by story), and the effect size is very large (Cohen's $d = 5.75$ by story), because the improvement

is nearly the same for every story. Figure 4.10 plots the two losses for every sample: every point lies below the diagonal, and the improvement is spread over all four prompt types and over easy and hard samples alike.

Table 4.5: Paired comparison of held-out loss: pretrained minus fine-tuned.

Quantity	**By sample**	**By story**
Number of pairs	191	49
Mean loss, pretrained	1.526	1.563
Mean loss, fine-tuned	1.189	1.222
Mean difference	0.336	0.341
95% bootstrap confidence interval	0.327 to 0.347	0.326 to 0.359
Pairs with lower fine-tuned loss	191 of 191	49 of 49
Cohen's d	4.56	5.75
Paired t statistic	63.00	40.22
p (paired t-test)	< 0.001	< 0.001
p (Wilcoxon signed-rank)	< 0.001	< 0.001

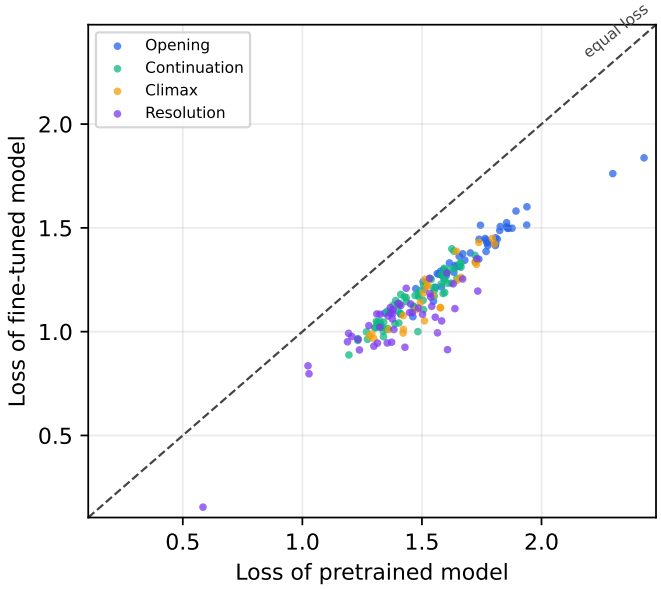


Figure 4.10: Held-out loss of each test sample under the two models. Points below the dashed line are samples on which the fine-tuned model is better.

4.1.3.2 Prompt Adherence of Generated Text

Table 4.6 compares the generated text of the two models on the measures of Section 3.3, and Figure 4.11 shows the distribution of lengths and the adherence rates.

The clearest difference is in the form of the answer. The pretrained model returned story text alone for 75% of the prompts. For the others it added something that the application has to remove: in 16 cases a title or heading before the story, usually in bold type, and in one case the line "To be continued" after it. The fine-tuned model returned story text alone for all 69 prompts, and the difference is significant (McNemar's exact test, $p < 0.001$).

The second difference is length, and here the pretrained model is better. All of its outputs were within 25% of the requested range, against 90% for the fine-tuned model ($p = 0.016$), and the fine-tuned segments were shorter by 69 words on average (Wilcoxon signed-rank test, $p < 0.001$).

The fine-tuned model used the protagonist's name in 88% of the 60 prompts that give one, against 97% for the pretrained model. This difference is not statistically significant ($p = 0.12$). In five of the seven fine-tuned outputs without the name, the name in the prompt was not a personal name at all but a word such as "Defenders" or "Guardian" that the keyword rules had picked from the test story by mistake. There was no difference between the models in complete endings ($p = 1.00$) or in lexical variety ($p = 0.84$).

Table 4.6: Prompt adherence of the text generated for 69 held-out prompts, with paired tests.

Measure	**Pretrained**	**Fine-tuned**	p
Length within 25% of the requested range	100%	90%	$= 0.016$
Story text only (no title, heading or comment)	75%	100%	< 0.001
Names the protagonist ($n = 60$)	97%	88%	$= 0.12$
Ends in a complete sentence	99%	99%	$= 1.00$
Words per segment, mean	344	275	< 0.001
Distinct word pairs, mean	0.948	0.948	$= 0.84$
Seconds per segment, mean	12.9	10.2	< 0.001

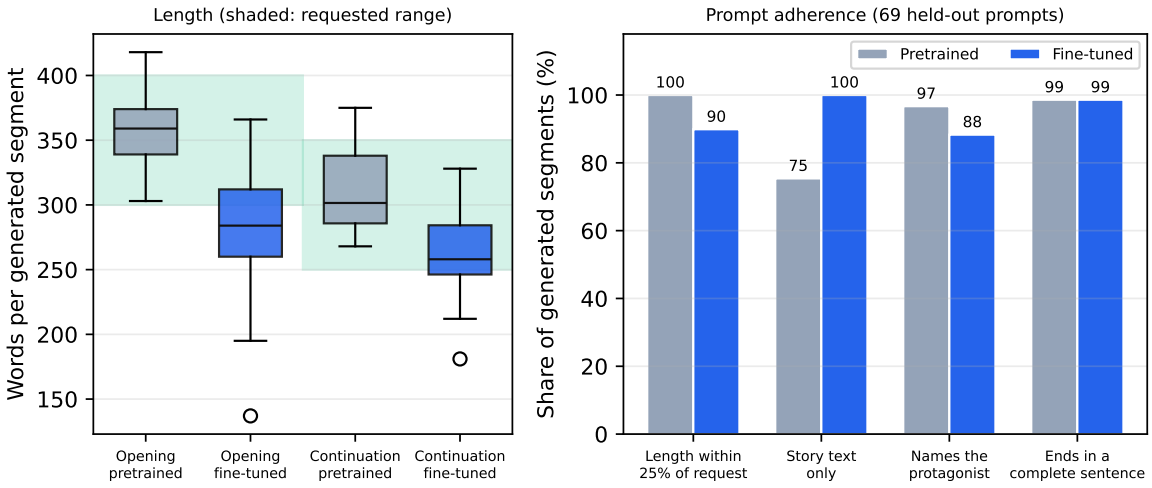


Figure 4.11: Length of the generated segments and prompt adherence for the pretrained and the fine-tuned model.

4.1.4 Summary of Findings

The main findings are as follows.

1. Fine-tuning reduced the held-out loss from 1.521 to 1.194 and the perplexity from 4.58 to 3.30 (28% lower). The loss fell for all four prompt types, for all 191 samples and for all 49 stories, and the difference is statistically significant.
2. Validation loss fell at every checkpoint, from 1.476 to 1.107, and was still falling after 600 steps.

3. The fine-tuned model always answered with story text alone (100% against 75%), with the same lexical variety and the same generation speed as the pretrained model.
4. The fine-tuned model wrote shorter segments than requested (275 words on average against 344), and it did not use the protagonist's name more reliably than the pretrained model.
5. On an 8 GB laptop the fine-tuned model writes a segment in about 10 s, and a full turn with choices took 22 to 28 s.
6. In the phase-1 user study, with a pretrained model, the framework was rated 4.05 for coherence, 4.12 for creativity, 4.35 for engagement and 3.75 for consistency, with an overall satisfaction of 4.3 out of 5; personalisation grew with the number of questions.

4.2 Discussion

4.2.1 Introduction

This section interprets the results, compares them with earlier work, and considers what they mean in practice, where the study is limited and what should be done next.

4.2.2 Interpretation of Results

The held-out comparison answers the central question of phase 2: a short low-rank adaptation on prompts in the system's own format makes the model fit this task better. The improvement is not confined to a few easy cases. It appears for every prompt type and for every test story, and it is large relative to the variation between samples. The model was trained on only 600 samples drawn from the 3,321 available and the validation loss was still falling, so the result shows what a small amount of training achieves, not the limit of the approach.

The generation benchmark shows what the model actually learned, and the picture is mixed. It learned the *form* of the answer completely: it never adds a title or a remark, which the pretrained model does in a quarter of the cases even

though the prompt ends with “Begin the story now”. This matters in practice, because every such addition has to be detected and removed before the text is shown. It also learned the style and the lengths of the training stories, and this is the source of the shorter segments. The training openings have 292 words on average, at the lower edge of the 300 to 400 words that the prompt asks for, and the other segments about 255. The fine-tuned model reproduces these lengths (281 and 260 words) and pays less attention to the number written in the prompt. The cause lies in how the stories were cut into segments, not in the method, and it can be corrected by cutting longer segments.

The use of the protagonist’s name points to the second weakness of the data. The answers attached to each training story come from keyword rules, and some of them are wrong; a group or an object is sometimes taken for the protagonist. A model trained on such pairs learns that the name in the prompt is not always the main character, and follows it a little less strictly. The difference from the pretrained model is not significant here, but its direction is a warning that label quality limits how much control fine-tuning can add.

Speed is unchanged because the adapter adds 2.62 million parameters to a model of 3.21 billion. A complete turn, with the request for choices, took under half a minute on a laptop with 8 GB of memory, with no network connection for the text.

The phase-1 ratings describe the framework and not the fine-tuned model. They show that the structured Q&A protocol is worth its cost: each additional group of questions raised personalisation, and coherence rose as well, which suggests that the extra information helps the model write a more focused story. The features rated highest were custom answers and illustrations, both of which are cheap to build. The weakest points were the meaningfulness of the offered choices and consistency in mystery stories, which depend on the model and on the simple rule-based checks.

4.2.3 Comparison with Previous Research

Long-form generators such as Re3 [24] and DOC [25], and the planning approach of Farrell and Ware [26], give a story strong structure, but they take their input once and then write on their own. The proposed system gets its structure from the five-phase model and still asks the user for a direction after every chapter. Co-writing tools such as Wordcraft [28] and Dramatron [30] give the user more freedom, but the user has to write or edit text; here the user only answers questions and picks directions, and in the phase-1 study custom answers were the most valued feature. Intelligent Grimm [31] produces better-matched images for a given story, while this system combines a simpler illustration step with narration and music around a generated story.

On the modelling side, the reviewed interactive systems use a general-purpose model without adaptation. The result here agrees with the wider evidence that low-rank adaptation is enough to specialise a large model [8] and that this can be done on quantised weights with modest hardware [9]: 0.082% of the weights, trained for an hour on a laptop, gave a clear and statistically significant improvement on held-out stories. The work differs from earlier story models trained on prompt-and-story corpora [6] in that the prompts are not free text written by people but the structured prompts of one application, produced by that application's own code.

4.2.4 Implications of the Findings

For developers of language-model applications, the main implication is that the prompt format of an application can be treated as a training target. When the prompts are produced by code, training data in exactly that format can be generated from existing texts without manual annotation, and a short adaptation run then fits the model to the application. The same procedure can be repeated when the prompts change. A second implication is that a hosted model is not required for an interactive product of this kind: a model with three billion parameters, adapted to the task and quantised, writes a story segment in well under half a

minute on a laptop, and the user's answers never leave the machine.

For educators, game designers and content creators, the phase-1 results indicate where effort pays off: users valued their own custom answers and the illustrations most, and personalisation grew with the number of questions up to about twelve. For researchers, the code, the dataset builder and the evaluation scripts [33] provide a baseline that can be reproduced on consumer hardware.

4.2.5 Limitations of the Study

The improvement in held-out loss was measured on stories from the same collection as the training data, so it shows that the model learned this kind of story from the system's prompts, not how it behaves on very different writing. The fine-tuned model has not yet been rated by users, because the user study (15 participants) was carried out earlier with a pretrained model. Scene illustrations depend on an online image service, which began to require payment during this work.

4.2.6 Recommendations for Future Research

The work can be taken forward in four directions:

- **User study of the fine-tuned model.** Participants should rate stories from the pretrained and the fine-tuned model without knowing which is which.
- **Better training data.** Longer story segments, labels from annotators instead of keyword rules, and training for more steps should correct the short segments and make the model follow every answer more strictly.
- **Local image generation.** A local model such as SDXL [17] would remove the dependence on the online image service.
- **Indian languages.** The same dataset and training scripts can be used to fine-tune the model for Hindi and Marathi stories.

Chapter 5

Conclusion

5.1 Conclusions

This dissertation set out to build a storytelling system in which the user shapes the story by answering questions and choosing directions, using a language model trained for the task and running on the user's own machine. The following conclusions can be drawn from the work.

A short structured questionnaire is a practical way to collect story preferences. Seventeen questions on setting, character, theme and plot, with a choice of 4, 8, 12 or 17 questions, were enough to produce openings that follow the user's answers, and user-added questions reached the story through the `custom_elements` channel without any change to the code. In the phase-1 user study personalisation rose from 3.2 to 4.6 out of 5 as the number of questions increased, and 12 questions gave the best balance between personalisation and answering time.

A small open model can be fine-tuned for this task on a laptop. Training 2.62 million of the 3.21 billion parameters of Llama 3.2 3B for about one hour reduced the loss on unseen stories from 1.521 to 1.194 and the perplexity from 4.58 to 3.30; the loss fell for every test sample and every test story, and the difference is statistically significant ($p < 0.001$).

Fine-tuning changed the form of the output more than its length. The fine-

tuned model always returned only the story text, whereas the original model added a title or a comment in a quarter of the cases, but its chapters were shorter than the prompt asked for (275 against 344 words on average) because the training segments were short. Generation speed stayed the same, at about 37 tokens per second, and a full turn with the four suggested directions took 22 to 28 seconds without a network connection.

The text, narration and music parts of the system run locally, but the scene illustrations depend on an online image service, which started to require payment during this work. A local image model is therefore needed before the whole system can run offline.

5.2 Contributions to Knowledge

1. A method for building fine-tuning data for a prompt-driven application: public texts are labelled with the application's own questions and passed through the application's own prompt code, so that training and use share one format.
2. A fine-tuned Llama 3.2 3B story model that runs locally, with a held-out comparison against the pretrained model using paired statistical tests, and a procedure for converting the trained adapter to the local runtime.
3. The `custom_elements` context channel, which passes user-added questions and answers to the model without any change to code or schema, and which was shown to work in a generated story.

5.3 Practical Implications

The language model of the system was trained and runs on a single ordinary laptop, using open components. For educators, this means personalised learning stories without sending pupils' answers to an outside service and without a cost per story. For game designers and content creators, the Q&A pattern is a way to collect rich preferences while keeping narrative control, and the fine-tuning procedure shows how to adapt a small open model to the house style of a product. For researchers and students, the open-source code, the dataset

builder and the evaluation scripts [33] form a baseline that can be reproduced in about two hours on consumer hardware, and the adapter can be retrained on other story collections or other languages with the same scripts.

Bibliography

[1] F. Khan, "Thousand stories, hundred genres (1k_stories_100_genre)." Hugging Face dataset, https://huggingface.co/datasets/FareedKhan/1k_stories_100_genre, 2023.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. arXiv:2005.14165.

[3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. arXiv:1706.03762.

[5] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[6] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 889–898, 2018.

[7] Meta AI, "Llama 3.2: Revolutionizing edge AI and vision with open, customizable models." https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024.

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36*, pp. 10088–10115, 2023.

[10] L. Yao, N. Peng, R. Weischedel, K. Knight, D. Zhao, and R. Yan, "Plan-and-write: Towards better automatic storytelling," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 7378–7385, 2019.

[11] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems 35*, pp. 27730–27744, 2022.

[12] G. Gerganov, "llama.cpp: LLM inference in C/C++." https://github.com/ggerganov/llama.cpp, 2023.

[13] Ollama Team, "Ollama: Run LLMs locally." https://ollama.ai/, 2024.

[14] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, "MLX: Efficient and flexible machine learning on Apple silicon." https://github.com/ml-explore/mlx, 2023.

[15] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10674–10685, 2022.

[17] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, "SDXL: Improving latent diffusion models for high-resolution image synthesis," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.01952.

[18] Pollinations.ai, "Free open image generation API." https://pollinations.ai/, 2024.

[19] W3C Speech API Community Group, "Web speech API specification (speech synthesis)," tech. rep., W3C Community Group Report, 2024.

[20] E. Casanova, K. Davis, E. Gölge, G. Göknar, I. Gulea, L. Hart, A. Aljafari, J. Meyer, R. Morais, S. Olayemi, and J. Weber, "XTTS: a massively multilingual zero-shot text-to-speech model," in *Interspeech 2024*, pp. 4978–4982, 2024.

[21] T. Brichta, "SoundHelix: Algorithmic music composition library." https://www.soundhelix.com/, 2024.

[22] P. Tambwekar, M. Dhuliawala, L. J. Martin, A. Mehta, B. Harrison, and M. O. Riedl, "Controllable neural story plot generation via reward shaping," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pp. 5982–5988, 2019.

[23] H. Rashkin, A. Celikyilmaz, Y. Choi, and J. Gao, "PlotMachines: Outline-conditioned generation with dynamic plot state tracking," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4274–4295, 2020.

[24] K. Yang, Y. Tian, N. Peng, and D. Klein, "Re3: Generating longer stories with recursive reprompting and revision," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 4393–4479, 2022.

[25] K. Yang, D. Klein, N. Peng, and Y. Tian, "DOC: Improving long story coherence with detailed outline control," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3378–3465, 2023.

[26] R. Farrell and S. G. Ware, "Large language models as narrative planning search guides," *IEEE Transactions on Games*, vol. 17, no. 2, pp. 419–428, 2025.

[27] N. Walton, "AI Dungeon: A text adventure game powered by deep learning." Latitude (online), 2019.

[28] A. Yuan, A. Coenen, E. Reif, and D. Ippolito, "Wordcraft: Story writing with large language models," in *27th International Conference on Intelligent User Interfaces*, pp. 841–852, 2022.

[29] J. J. Y. Chung, W. Kim, K. M. Yoo, H. Lee, E. Adar, and M. Chang, "TaleBrush: Sketching stories with generative pretrained language models," in *CHI Conference on Human Factors in Computing Systems*, pp. 1–19, 2022.

[30] P. Mirowski, K. W. Mathewson, J. Pittman, and R. Evans, "Co-writing screenplays and theatre scripts with language models: Evaluation by industry professionals," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–34, 2023.

[31] C. Liu, H. Wu, Y. Zhong, X. Zhang, Y. Wang, and W. Xie, "Intelligent grimm - open-ended visual storytelling via latent diffusion models," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6190–6200, 2024.

[32] S. Colvin, "Pydantic v2: Data validation using Python type hints." https://docs.pydantic.dev/, 2024.

[33] A. M. Jha, "AI-Story-Generator: An open-source Q&A-driven multimodal storytelling framework." https://github.com/DeepPatange/AI-Stroy-Generator, 2026.

Appendices

Appendix A

List of Publications

[1] Anjali Jha, Vidyullata Devmane, "Q&A Based Interactive Storytelling Using Generative AI", accepted for presentation at the 2nd International Conference on Computational Technology for Research in Data Analytics (ICCTRDA 2026), organised by Duy Tan University, Vietnam, November 26–28, 2026, publication in Springer's Lecture Notes in Networks and Systems (LNNS) series. **[Accepted]**

Appendix B

Acceptance Letter

The acceptance mail for the paper listed in Appendix A, received from ICCTRDA 2026, is given below.

Fwd: ICCTRDA 2026: Paper Notification 275
1 message

Madan Anjali <madan.anjali24@sakec.ac.in> Sat, 29 Aug, 2026 at 17:12
To: Anjali Jha <jhanilu1998@gmail.com>

---------- Forwarded message ---------
From: **Madan Anjali** <madan.anjali24@sakec.ac.in>
Date: Fri, 21 Aug, 2026, 18:59
Subject: Fwd: ICCTRDA 2026: Paper Notification 275
To: <shashikant.radke@sakec.ac.in>

---------- Forwarded message ---------
From: **ICCTRDA - 2026@Vietnam** <icctrda.congress@gmail.com>
Date: Wed, 19 Aug, 2026, 14:45
Subject: ICCTRDA 2026: Paper Notification 275
To: Madan Anjali <madan.anjali24@sakec.ac.in>

International Conference on Computational Technologies for Research in Data Analytics 2025: ICCTRDA 2026

Dear **Author(s),**

Greetings from **ICCTRDA 2026!**

ICCTRDA-2026 team is pleased to inform you that your paper with submission ID **275** and Paper Title '**Q&A-Driven Interactive Storytelling Using Large Language Models: A Multimodal Framework for Personalised Narrative Generation**' has been accepted for presentation at "ICCTRDA2026" and for publication in the conference proceedings. The Committee thanks you for your contribution.

The conference proceedings will be published by Springer in Lecture Notes in Networks and Systems series [Indexing: SCOPUS, INSPEC, WTI Frankfurt eG, zbMATH, SCImago; All books published in the series are submitted for consideration in Web of Science]. This acceptance means that your paper is among the top 15% of the papers received/reviewed. The registrations for the conference are open. **We want to provide you with urgent information and advise you that we have limited slots available, and once they are filled, we will not be able to accommodate any further registrations. To secure your spot at this highly anticipated event, we urge you to complete your registration without delay.**

You are requested to do the registration as soon as possible and submit the following documents to **icctrda.congress@gmail.com** at the earliest.
1. Final Camera-Ready Copy (CRC) as per the springer format. (See https://www.icctrda.com/downloads)
2. Copy of e-receipt of registration fees. (For Registration, see https://www.icctrda.com/registrations)
3. The final revised copy of your paper should also be uploaded via Microsoft CMT.

The reviewers comments are given at the bottom of this letter, please improve your paper as per the reviewers comments along with the following suggestions.
1. The dataset link must be presented in the paper (if used in the manuscript)
2. Implementation details must be presented in the paper.
3. All references must have complete information like Authors, Title, Conference/Journal name, Volume/issue/pages/doi, and year.
4. All references must be checked for validity. Self citations is not allowed (until relevant)
5. Please proofread the manuscript for tortured/paraphrased sentences before submitting the final paper.
6. All Equations and tables must be in editable form (not images) if any
7. The paper must be in single column format (minimum 8-pages)

The paper prior to submission should be checked for plagiarism and AI Plagiarism from licensed plagiarism softwares like Turnitin/iAuthenticate etc. The similarity content should not exceed 15% and AI similarity should not exceed 5%.

Pay registration fees via online portal:
[Kindly note – the conference being organised in Hybrid Mode and you can choose the mode of presentation in either physical (offline) or digital (online) mode; then pay the registration fees]
https://www.icctrda.com/registrations

Once you pay the registration fees, kindly fill the following google form:
https://forms.gle/jGqMXoKqEjdQj1UJ9

With Regards
Conference Chair

Reviewer-1
1. Restrict the paper page length between 10-12 pages. 2 conceptualization and formal analysis written well. 3 theorem proving and supported lemmas explained well in support with algorithm. 4 result and analysis discussed. 5 how performance metrics being validated is not clear. 6 conclusion has future sscope and limitations. 7 formatting of pper must be done as per springer conference template. 8 cite all references in text 9 . validation and template matching is well justified. 10 proof read entire paper once.

Reviewer-2
Paper is nice but it requires restructuring as follows:
1. The paper must be started with introduction section including motivation, main contributions and organization of paper.
2. Literature review section is missing/can be extended.
3. Proposed methodology includes a flow chart describing complete proposed work, algorithm, dataset information
4. results and discussion section includes the results of the proposed method and its related discussion
5. comparison with existing literature must be included\
6. the paper ends with a conclusion with limitation of work and future scope
7. Figures are rarely visible; please make it more bigger and visible.
8. Algorithm must not in figures
9. Language must be improved.
10. The paper limit must be within 12 pages. (no more than 12 pages & a minimum of 8 papers)
11. Avoid snapshot as results; it should be in graph form.
12. Please check the reference for validation, some references requires full information like author name, title, journal/conference, volume, issue and page number, DOI.

Please don't print this email, unless you need to.

Vision:

To become a globally recognized institution offering quality education and enhancing professional standards.

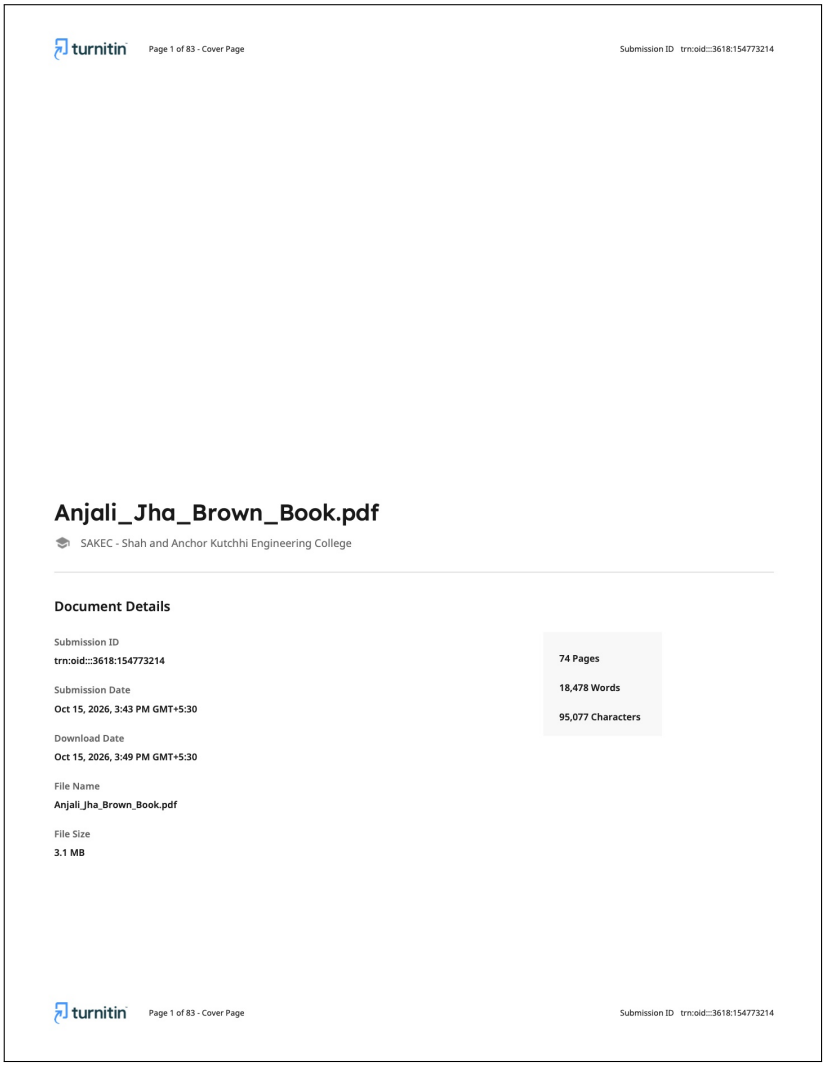
Mission:

To impart high quality technical education to the students by providing an excellent academic environment, well-equipped laboratories and training through the motivated teachers.

Appendix C

Plagiarism Report

The similarity of this report was checked with Turnitin on 15 October 2026. The overall similarity is 9%, below the acceptable limit of 10%. The cover page and the integrity overview of the Turnitin report are given below.



turnitin Page 1 of 83 - Cover Page Submission ID trn:oid:::3618:154773214

Anjali_Jha_Brown_Book.pdf

SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID
trn:oid:::3618:154773214

Submission Date
Oct 15, 2026, 3:43 PM GMT+5:30

Download Date
Oct 15, 2026, 3:49 PM GMT+5:30

File Name
Anjali_Jha_Brown_Book.pdf

File Size
3.1 MB

74 Pages

18,478 Words

95,077 Characters

turnitin Page 1 of 83 - Cover Page Submission ID trn:oid:::3618:154773214

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 8 words)

Match Groups

105 Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks

12 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
3% Publications
6% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Hidden Text
816 suspect characters on 4 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.